Technische Universität Dresden

Faculty of Computer Science, Chair of Systems Engineering

A Comparative Analysis of WebAssembly and Docker Containers for Microservice Architectures in Kubernetes

Abdulaziz Musaev

Born on 22nd November, 1999 in Denau, Uzbekistan

Master's Thesis in Distributed Systems Engineering

28 May 2026

Advisors

Dr. K. Pubudu N. Jayasena

Dr. Michael Steininger (IAV GmbH)

Referees

Dr. K. Pubudu N. Jayasena

Prof. Dr. Christof Fetzer

Supervising professor

Prof. Dr. Christof Fetzer

Erklärung

Ich versichere, dass ich alle bewertungsrelevanten Beiträge dieser Arbeit ohne Hilfe anderer Personen und Werkzeuge erbracht habe oder diese Beiträge klar und unmissverständlich im Text gekennzeichnet habe (etwa durch Zitate). Diese Arbeit wurde in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt. Sollte ich mehrere Kopien dieser Arbeit zur Bewertung abgeben, so ist deren Inhalt identisch.

Declaration

I declare that I have achieved all grading-relevant contributions to this work without the help of other persons or tools, or that I have clearly and unmistakably identified such contributions in the text (e.g., by citations). This work has not been submitted in the same or a similar form to any other examination body. Should I submit multiple copies of this work for grading, their content is identical.

(Abdulaziz Musaev)
Dresden, 28 May 2026

Acknowledgments

I would like to express my sincere gratitude to **IAV GmbH**, the company at which this master's thesis was carried out, for providing me with the opportunity, the technical environment, and the supportive atmosphere in which to pursue this research.

I am especially grateful to **Mr. Rico Friedrich**, my project manager at IAV GmbH, who helped me tremendously in shaping a relevant and feasible thesis topic. From our very first conversation he took my personal interest in the Rust ecosystem and WebAssembly seriously and guided me toward a research direction that turned out to be both academically meaningful and industrially relevant.

I am equally indebted to my company supervisor, **Dr. Michael Steininger**, for his patient guidance from the first day of the project through to its completion. His kindness, helpfulness, and understanding made every stage of this work substantially easier; without his consistent advice and support, this thesis would not have been possible.

Finally, my heartfelt thanks go to my two university professors, **Prof. Dr. Christof Fetzer** and **Dr. K. Pubudu N. Jayasena**, for their academic supervision and the formal review of this work. Their feedback shaped both the framing of the research questions and the quality of the final presentation.

Abstract

WebAssembly (Wasm) is a portable, sandboxed binary instruction format originally designed for web browsers that has recently emerged as a server-side execution environment for cloud-native workloads. With the WebAssembly System Interface (WASI) and the Component Model standardised in WASI Preview 2 (P2), Wasm modules can be deployed alongside Docker containers in Kubernetes clusters via the CNCF-sandbox SpinKube stack, raising the practical question of whether server-side Wasm is, today, a viable alternative to Docker for production HTTP microservices.

This thesis presents a controlled side-by-side comparison of Docker and WebAssembly microservices in upstream Kubernetes (kubeadm v1.34) on a dedicated Hetzner Cloud ccx23 node. Four primary variants are evaluated in a 2×2 matrix that crosses the runtime axis (Docker/`runc` vs. WebAssembly/SpinKube) with the language family axis (Rust vs. Go-family): `wasm-rust`, `wasm-tinygo`, `docker-rust`, and `docker-golang`. The Wasm variants run on Fermyon Spin + Wasmtime/Cranelift via the `containerd-shim-spin-v2` shim and dispatch HTTP requests through the standardised `wasi:http/incoming-handle` interface. Two complementary workloads—a CPU-bound prime sieve and a memory-bandwidth benchmark with SHA-256—are measured under both single-worker (*limited*) and multi-worker (*unlimited*) configurations across five quantitative metrics (OCI artifact size, cold- and warm-start latency, throughput under k6-driven concurrent load, end-to-end latency percentiles, and memory/CPU footprint) and a four-dimension qualitative developer-experience assessment (toolchain setup, dependency compatibility, Kubernetes integration, debugging and observability).

The results confirm Wasm's structural advantages on the artifact axis: Wasm OCI images are dramatically smaller than their Docker counterparts—most pronounced for the `wasm-rust` path—and cold-start density on a Kubernetes node is consequently higher. Under concurrent HTTP load, however, the picture inverts. The server-side Wasm ecosystem still lacks first-class async and multi-threading primitives, and the request-per-instance dispatch model of Spin under WASI P2 keeps each component effectively single-threaded; the Wasm variants therefore sit substantially behind the Docker baselines on sustained throughput in the limited configuration. `docker-golang` achieves the highest sustained throughput, followed by `docker-rust` and `wasm-rust`, with `wasm-tinygo` lagging most noticeably because TinyGo's `-target=wasip2` cannot export `wasi:http/incoming-handler` and the variant must therefore use `-target=wasip1` through a CGo-based Spin SDK bridge—a structural incompatibility that compounds the concurrency gap with toolchain-level friction. End-to-end p95 latency follows the same ordering. These findings are consistent with the prior peer-reviewed work surveyed in the related-work chapter on Wasm concurrency under WASI P1, Software-Based Fault Isolation (SFI) bounds-checking overhead, and the partial mitigation provided by the WASI P2 transition—specifically, the request-handler model and instance pooling that allow interleaved request processing without yet enabling intra-component multithreading.

The qualitative developer-experience assessment reaches the same conclusion: while dependency compatibility sits at parity with Docker on the SpinKube path, toolchain setup, Kubernetes integration, and—especially—live debugging require materially more effort on the Wasm side, and the operational cost of getting a Wasm service to production-grade reliability remains substantially higher than for an equivalent Docker container. The thesis concludes that, notwithstanding Wasm's clear wins on image size and cold-start density, **Docker remains the more mature and productive default for general-purpose microservice architectures in Kubernetes today**; server-side Wasm with SpinKube/WASI Preview 2 is a strong choice for narrow workload classes—stateless, low-concurrency, density-critical—where its specific structural advantages outweigh its current ecosystem limitations.

Contents

Chapter 1

Introduction

1.1 Context

The modern cloud computing landscape relies heavily on microservices and containerization to deliver scalable, resilient applications. Docker and the Open Container Initiative (OCI) standards, frequently paired with mature backend languages like Golang, have become the de facto foundation for these deployments within Kubernetes (K8s) environments [1, 2]. This traditional cloud-native stack prioritizes a frictionless developer experience and mature tooling. However, the growing demand for edge computing, high-density serverless functions, and instantaneous scaling has exposed the architectural limitations of conventional Linux containers. OCI containers rely on operating-system-level virtualization, which carries a baseline overhead in both memory consumption and initialization time [3].

As organizations seek to maximize hardware utilization and minimize latency, a shift toward more lightweight execution environments is underway. WebAssembly (Wasm), originally designed as a highly optimized, stack-based virtual machine binary format for web browsers [4], has emerged as a potential solution for server-side workloads. With the introduction of the WebAssembly System Interface (WASI), Wasm promises a portable, secure, and near-instantaneous execution model that operates without the heavy dependencies of a traditional Linux container [5]. As illustrated in Figure 1.1, this thesis evaluates four implementation variants spanning two runtime paths within a single K8s cluster.

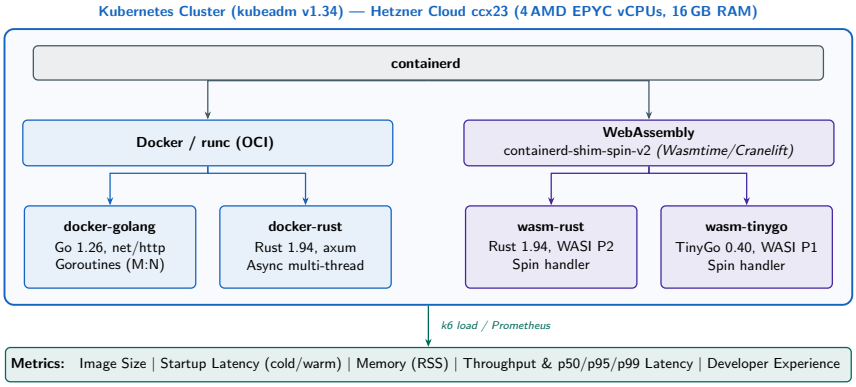


Figure 1.1 – Experimental setup: four implementation variants across two runtime paths within a Kubernetes cluster, with the five metrics collected in this study. Source: Author's own illustration

1.2 Problem Statement

Standard OCI containers introduce measurable overhead regarding memory footprint and cold-start latency. While negligible for traditional monolithic applications, this overhead becomes a severe limiting factor for transient microservices that require rapid, sub-millisecond scaling. The emerging Wasm stack theoretically solves this by utilizing Software-Based Fault Isolation (SFI) to strip away the operating system layer entirely, offering drastically reduced image sizes and high-density execution [6].

However, transitioning from a mature ecosystem to a nascent one introduces substantial, often under-documented friction. The core problem this thesis addresses is the tension between computational efficiency and developer velocity. While the Wasm ecosystem promises high performance for server-side workloads, its current WASI Preview 1 (P1) runtime environment does not expose multi-threaded Hypertext Transfer Protocol (HTTP) primitives, constraining concurrent request handling to single-threaded execution models [5, 7]. Furthermore, libraries that depend on Operating System (OS)-specific features absent from WASI P1—such as Portable Operating System Interface (POSIX) thread spawning, native networking sockets beyond runtime-specific extensions, and C++ exceptions—cannot be directly compiled to Wasm, limiting the available dependency ecosystem [8, 9].

There is a critical need to empirically quantify the holistic trade-offs of these two paradigms. It must be determined whether the theoretical efficiency gains of the Wasm stack outweigh the practical performance degradations and the operational complexities it introduces compared to the industry-standard Docker baseline.

1.3 Research Questions

To address the problem statement, this thesis investigates the following primary research questions:

- **RQ1:** How does the memory footprint and OCI artifact size of the SpinKube/Wasmtime Wasm stack compare to a Docker/`runc` baseline in a single-node K8s cluster?
- **RQ2:** To what extent do Wasm runtimes improve cold- and warm-start latency compared to OCI containers executed under `runc`, across both garbage-collected (Go) and ahead-of-time-compiled (Rust) baselines?
- **RQ3:** What are the runtime performance trade-offs—specifically request throughput and end-to-end latency—introduced by SpinKube's WASI P2 single-component dispatch model (one request per Wasmtime instance at a time) compared to Docker/`runc`-based concurrency models?
- **RQ4:** How do the current Wasm-on-Kubernetes ecosystem (SpinKube, the WASI Preview 1 / Preview 2 toolchains, library compatibility) and operational integration impact developer experience compared to a mature Docker/K8s workflow?

1.4 Research Methodology

This research employs a mixed-methods approach, combining quantitative performance benchmarking with a qualitative assessment of developer experience.

Rather than isolating the runtime—which would necessitate comparing equivalent languages and thereby destroying the ability to measure real-world developer experience—this study compares

two dominant paradigms in their native contexts using four implementation variants. A baseline microservice is developed using Golang and compiled into a standard Linux-based Docker image, and a second baseline uses Rust with Docker to partially deconfound the language variable. Two experimental variants compile functionally equivalent services to WASI-compliant Wasm modules: one using Rust and one using TinyGo. All four artifacts are deployed into an isolated K8s test cluster utilizing `containerd`, standard `runc`, and a Wasm shim.

Quantitative data is gathered by subjecting all deployments to standardized load testing. Monitoring tools measure cold-start times, memory consumption, CPU utilization, throughput, and request latency. The approach acknowledges that language-level differences (e.g., Go's garbage collection versus Rust's manual memory management) influence the metrics, but is designed to reflect the real-world performance delta an engineering team experiences when transitioning between these technological stacks. A qualitative analysis is also conducted on the development lifecycle, documenting compilation friction, library compatibility, and pipeline integration complexity.

1.5 Thesis Contributions

This thesis provides the following key contributions to the field of cloud-native architecture:

1. **Holistic Paradigm Benchmark:** A side-by-side performance comparison of the Golang/Docker stack versus the Rust/Wasm stack, highlighting specific metrics where the Wasm ecosystem succeeds (density, initialization) and where it currently falls short (throughput, concurrent request handling) due to WASI P1 ecosystem constraints.
2. **Developer Experience Assessment:** A critical evaluation of the current state of Wasm for backend development, quantifying the engineering friction of compiling Rust and TinyGo code to WASI and navigating incompatible library dependencies compared to standard Go and Docker development.
3. **Adoption Framework:** Actionable recommendations for cloud architects, delineating specific use cases where the Wasm stack is currently viable and identifying the operational risks that make it unsuitable for rapid-iteration product teams accustomed to the Go/Docker ecosystem.

1.6 Thesis Structure

The remainder of this thesis is structured as follows:

- **Chapter 2: Background** provides the foundational technical context on cloud computing paradigms, containerization technologies, and the evolution of Wasm and WASI—including the Component Model, available runtimes, and the K8s integration landscape.
- **Chapter 3: Related Work** surveys peer-reviewed performance studies comparing server-side Wasm and container workloads, security analyses, and compiler strategy trade-offs, and additionally surveys the commercial deployment landscape of server-side Wasm and concrete production deployments.
- **Chapter 4: Research Methodology** defines the comparative analysis strategy, establishes the four implementation variants and their selection rationale, and outlines the precise measurement metrics.

- **Chapter 5: Implementation** details the cloud infrastructure setup, benchmark service design, all four service implementations, and the K8s cluster configuration for hybrid Wasm and Docker workloads.
- **Chapter 6: Evaluation** presents the quantitative benchmark results for the prime sieve benchmark alongside the qualitative assessment of developer experience.
- **Chapter 7: Discussion and Future Work** analyzes the trade-offs between technological maturity, computational efficiency, and developer velocity, and outlines planned future experiments.
- **Chapter 8: Conclusion** summarizes the core findings and final verdicts of the research.

Chapter 2

Background

This chapter provides the foundational technical context required to evaluate the comparative analysis presented in this thesis. It traces the evolution of computing infrastructure from on-premises hardware to modern cloud paradigms, details the mechanics of containerization and orchestration, and culminates in a comprehensive architectural breakdown of Wasm as an emerging server-side execution environment. This background is organised into three sections: cloud computing paradigms and microservice architecture (§2.1), containerization and Kubernetes orchestration (§2.2), and the technical foundations of WebAssembly and WASI—including the Component Model, available runtimes, and the K8s integration landscape (§2.3). Industry adoption and production deployments of server-side Wasm are surveyed in Chapter 3.

2.1 Cloud Computing and Microservices

This section establishes the cloud computing context that motivates the need for lightweight execution environments studied in this thesis. It traces the paradigm evolution from Infrastructure as a Service (IaaS) to Function as a Service (FaaS) and then examines microservice architecture as the software design pattern that drove the demand for efficient, low-overhead container runtimes.

2.1.1 The Evolution of Cloud Paradigms

The genesis of modern software deployment traces back to on-premises data centers, where organizations were solely responsible for the procurement, maintenance, and scaling of physical hardware. This capital-heavy model suffered from severe resource underutilization; servers were provisioned for peak load, leaving them idle during standard operations. The modern cloud computing era shifted the industry to a pay-as-you-go, scalable operational model, fundamentally altering how infrastructure is managed [10].

As cloud computing matured, the industry developed a “Shared Responsibility Model,” where the abstraction layer moved progressively higher up the stack. As illustrated in Figure 2.1, this spectrum dictates whether the cloud provider or the client is responsible for specific layers of the technological stack:

- **Infrastructure as a Service (IaaS):** The cloud provider manages the physical hardware, storage, and networking. The client rents virtualized infrastructure (e.g., a Virtual Machine (VM)) and retains full responsibility for managing the operating system, middleware, language runtimes,

and the application code itself. This offers maximum control but requires significant operational overhead.

- **Platform as a Service (PaaS):** The provider abstracts both the underlying hardware and the operating system. The client is provided a managed environment and is solely responsible for deploying their application code and managing their data. Traditional Docker container deployments often operate in this space.
- **Function as a Service (FaaS) / Serverless:** The highest level of *infrastructure* abstraction for a developer. The cloud provider dynamically manages the allocation, provisioning, and scaling of the servers and runtimes. The client writes granular, event-driven functions and dictates the trigger configurations. Crucially, FaaS allows applications to "scale to zero," eliminating compute costs during periods of inactivity [11].
- **Software as a Service (SaaS):** The highest level of *software* abstraction. The cloud provider hosts and manages the entire application stack. The client manages nothing regarding the deployment or execution environment; they simply consume the finished software over the internet.

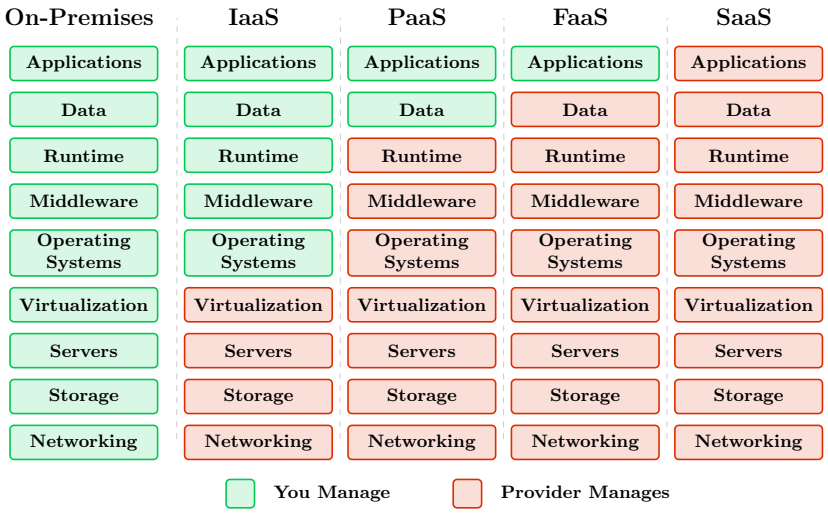


Figure 2.1 – The Shared Responsibility Model across varying Cloud Computing Paradigms. Adapted from Mazzeschi [12]

For the purposes of this research, SaaS falls outside the scope of architectural implementation. The critical transition lies between PaaS and FaaS. The relentless industry drive toward rapid elasticity, microsecond billing, and extreme resource efficiency in FaaS and edge environments exposed the limitations of traditional, heavy deployment artifacts. Standard OCI containers, laden with operating system userlands, introduce severe latency bottlenecks in these paradigms, necessitating a shift toward the lightweight isolation models that Wasm provides.

2.1.2 Microservice Architecture

Microservice architecture emerged as the logical software design pattern for cloud-native deployment. Unlike traditional monolithic applications—where all business logic, user interfaces, and database

connections are tightly coupled into a single executable—microservices decompose an application into a collection of small, independently deployable, and loosely coupled services that communicate over a network via language-agnostic Application Programming Interfaces (APIs) [13].

This paradigm offers distinct advantages: development agility, localized fault isolation, and the ability to scale specific system bottlenecks independently. However, it introduces severe distributed system challenges. Managing network latency, data consistency, and the operational overhead of packaging and deploying hundreds of individual services necessitated the creation of entirely new virtualization and orchestration layers [2, 13].

The relevance of microservice architecture to this thesis is direct: the experimental benchmark in Chapter 4 implements a stateless HTTP microservice deployed in four variants within a Kubernetes cluster. The throughput and latency differences between the WASI Preview 2 (P2) request-handler model (SpinKube) and Docker's goroutine/async model are most acutely visible under concurrent request loads, making microservices the natural evaluation domain for this comparison.

2.2 Containerization and Orchestration

This section covers the containerization technologies and orchestration platform that form the deployment baseline compared in this thesis. It first contrasts Virtual Machine (VM) with containers to motivate Docker's design, then covers Docker's OCI standardization, and finally describes how K8s orchestrates containerized workloads at scale—the same platform used to run all four benchmark variants.

2.2.1 Virtual Machines vs. Containers

To solve the deployment complexity of microservices, the industry initially relied on VMs. However, VMs require a hypervisor to emulate physical hardware, atop which a complete guest OS is installed. This results in gigabytes of memory overhead and startup times typically measured in tens of seconds to minutes [1], making them inherently unsuitable for transient, rapidly scaling microservices.

Docker revolutionized the landscape by democratizing Linux containers. Instead of virtualizing the hardware, containers virtualize the operating system [1]. As shown in Figure 2.2, Docker relies on two core Linux kernel features:

1. **Namespaces:** Provide process isolation, ensuring the container has its own isolated filesystem, network interfaces, and process tree.

2. **Cgroups (Control Groups):** Limit, isolate, and monitor the resource usage (CPU, memory, disk I/O) of a given process.

Docker's definitive innovation was the standardization of the container image format, now governed by the OCI [14]. By packaging application code alongside its dependencies, libraries, and a base filesystem into a single portable unit, Docker ensured environment consistency from a developer's laptop to production servers [1].

Despite its dominance, the OCI standard has inherent drawbacks. Because containers share the host's Linux kernel, a kernel-level vulnerability can potentially compromise all containers on that host, posing security risks in multi-tenant environments. Furthermore, traditional container images are bulky; they bundle a userland OS and language runtimes alongside the application. This bulk introduces

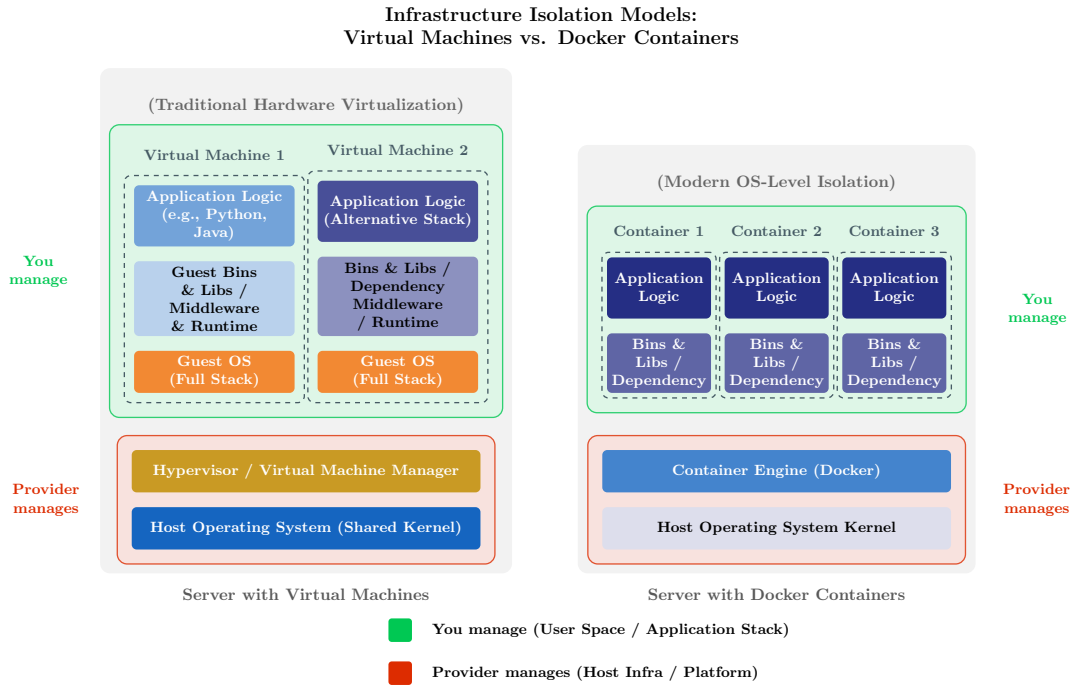


Figure 2.2 – Infrastructure Isolation Models: Virtual Machines vs. Docker Containers

latency during image transfer and initialization—known as the "cold start" problem—which is highly detrimental in FaaS and edge computing environments [6].

2.2.2 Kubernetes Orchestration

As microservices proliferated, manually managing individual containers across a fleet of servers became operationally unfeasible. K8s emerged as the industry-standard container orchestration platform. It abstracts the underlying cluster of hardware, allowing engineers to declare the desired state of their applications rather than manually scripting deployments [2].

K8s operates on a distributed architecture, as depicted in Figure 2.3. The Control Plane dictates instructions to worker nodes. Each worker node runs an agent called a Kubelet, which manages Pods—the smallest deployable computing unit, typically containing one or more tightly coupled containers [2]. Crucially, K8s communicates with the underlying container runtime (like containerd or CRI-O) via the Container Runtime Interface (CRI). This decoupled architecture established strict standardizations, eventually allowing alternative, non-Linux execution environments, such as Wasm, to be orchestrated seamlessly alongside standard OCI-compliant containers.

2.3 WebAssembly

This section provides the technical foundation for WebAssembly (Wasm) and its server-side system interface (WASI), the execution environment used by the two Wasm variants in this study's 2×2 benchmark matrix. Wasm's design was shaped by its browser origins; understanding this context is essential to appreciating both its advantages—portability, compact binaries, and structural memory isolation—and the current limitations of its server-side deployment model that this thesis investigates.

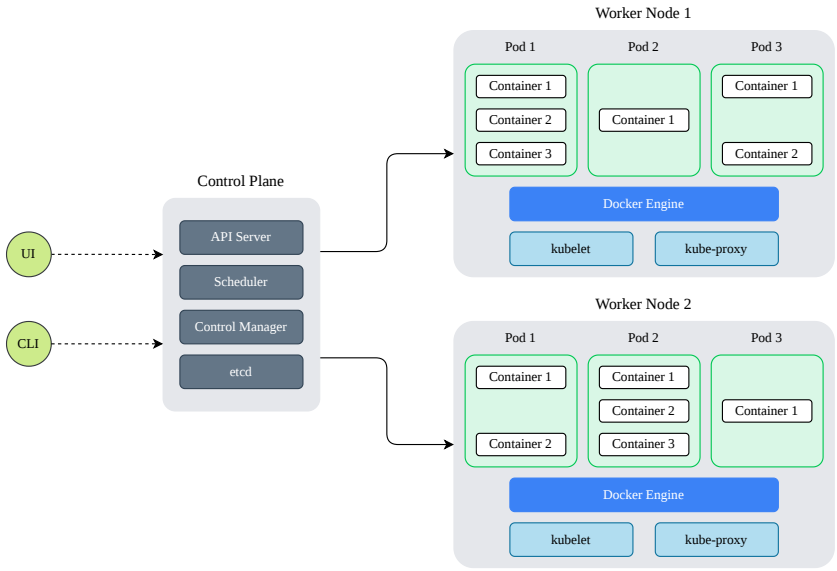


Figure 2.3 – Kubernetes Architecture: Control Plane, worker nodes, Pods, and the CRI decoupling layer

This section begins with Wasm's origins and technical architecture, covers its module format, memory model, language support, and security model, then describes the WASI interface layer, the Component Model evolution that defines the future direction of this research, the current industry adoption landscape, available runtimes, and the K8s integration mechanisms used in this thesis.

2.3.1 Origin and Technical Architecture

Wasm is a binary instruction format designed for a stack-based virtual machine. It originated from the need to execute high-performance code within web browsers—a domain historically constrained by JavaScript's inconsistent performance and parsing overhead. Wasm was officially released in 2017 as a World Wide Web Consortium (W3C) standard, sitting alongside HyperText Markup Language (HTML), Cascading Style Sheets (CSS), and JavaScript as an official language of the web [4]. This browser heritage established the two properties most consequential for this thesis: its strict isolation model, designed to prevent malicious web code from accessing the host system, and its compact binary format, optimised for fast transmission over network connections. Developers write code in systems languages like C, C++, or Rust and compile it down to `.wasm` binaries, as illustrated in Figure 2.4. Beyond these systems languages, Wasm serves as a compilation target for more than 40 programming languages [15], including Go, Python, Kotlin, C#, and TypeScript, with varying degrees of ecosystem maturity across WASI runtimes.

2.3.2 Module Structure and Stack-Based Execution

This subsection describes how Wasm programs are structured and executed. These mechanical properties—stack-based execution, compact binary sections, and ahead-of-time or just-in-time compilation—directly govern the startup latency and compute performance metrics measured in this thesis.

Wasm is fundamentally a stack machine [4]. Unlike register-based physical CPU architectures (such as x86_64 or ARM64), Wasm instructions implicitly operate on an operand stack. Variables are pushed

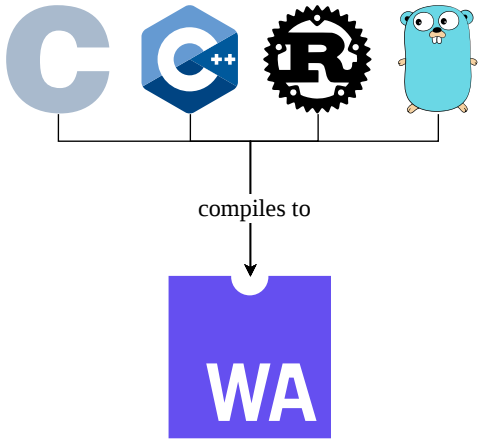


Figure 2.4 – A selection of programming languages that compile to Wasm, illustrating its language-agnostic compilation model

onto the stack, and operations pop these operands, compute the result, and push it back. This design yields highly compact binary modules, as instructions do not need to explicitly encode register numbers, accelerating both transmission over networks and decoding phases.

A Wasm application is distributed as a strictly standardized binary module consisting of discrete sections [4]. These include the Type section (function signatures), Import section (host-provided dependencies), Code section (executable bytecode), and Data section (initial memory state). Before execution, a Wasm runtime performs a deterministic, single-pass validation to ensure type safety and structural integrity. Because the bytecode is designed to map cleanly to native machine instructions, runtimes employ Just-In-Time (JIT) or Ahead-of-Time (AOT) compilation engines (such as Cranelift or LLVM) to translate Wasm into highly optimized, native machine code instantaneously [16].

2.3.3 Memory Management and Isolation

This subsection explains Wasm's approach to memory safety and fault containment—the properties that make it a compelling alternative to kernel-based container isolation for multi-tenant deployments and that are analysed in the security comparison in Chapter 3.

One of Wasm's defining technical characteristics is its approach to memory management and fault isolation. The architecture relies on the following core principles:

- **Compact Binary Format:** The `.wasm` format is highly optimized, allowing for rapid fetching, decoding, and execution [4].
- **Hardware Agnosticism:** Because it utilizes a virtual stack machine, the compiled bytecode makes no assumptions about the underlying physical CPU architecture, enabling true portability across varied cloud node pools [4].
- **Linear Memory Space:** Wasm operates in a contiguous, resizable array of bytes known as linear memory [4]. Code executing within the Wasm module operates strictly with integer offsets within this array; it cannot execute arbitrary pointers or access host memory outside this strictly enforced boundary.

- **Software-Based Fault Isolation (SFI):** Unlike Docker, which relies on the host OS kernel (cgroups/namespaces) to contain a process, Wasm enforces isolation through a combination of static validation and runtime bounds-checking [4, 17]. At load time, the runtime performs a single-pass type-safety check; at execution time, every memory access is confined to the module's linear memory region by compiled-in bounds checks. Any attempt to access memory outside the allocated linear boundary results in a deterministic trap, halting the module instantly without compromising the host.

2.3.4 Language Agnosticism and the JVM Comparison

The concept of Write Once, Run Anywhere (WORA) was popularized by Java. Java achieves portability by compiling source code to Java bytecode, which is executed by the Java Virtual Machine (JVM) [18]. However, Wasm fundamentally differs from the JVM in design and scope, as illustrated in Figure 2.5. The JVM was built specifically to execute Java and inherently carries a heavyweight runtime featuring built-in garbage collection. Conversely, Wasm was designed as a low-level, truly language-agnostic compilation target without an opinionated runtime, making it exceptionally lightweight [4].

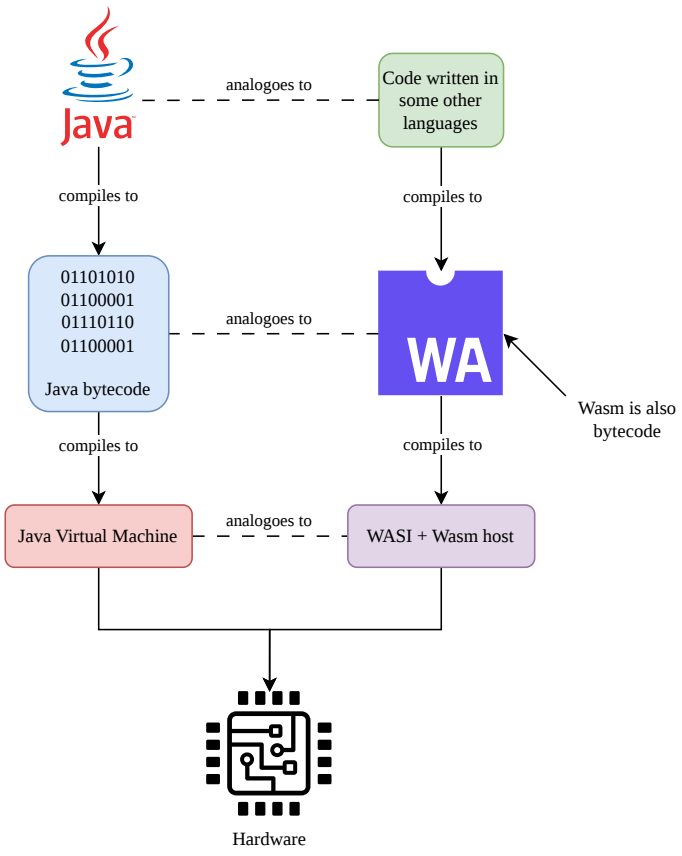


Figure 2.5 – An analogy comparing the cross-platform execution models of Java (JVM) and Wasm. Adapted from Chiarlone [18]

For languages requiring automatic memory management (like Go, Python, Kotlin, or PHP), Wasm historically required the entire language runtime and garbage collector to be compiled and bundled

inside the `.wasm` binary. This approach bloated the deployment artifact and created redundant layers of memory management. For instance, deploying a compiled-to-Wasm language with its own garbage collector into a host environment that already possesses one (such as a browser engine like V8) is highly wasteful [19]. Furthermore, having no interaction between Wasm's linear memory and the built-on-top garbage collector of the ported language often caused problems like memory leaks and inefficient collection attempts [20].

To resolve this, the WebAssembly Garbage Collection (WasmGC) standard has been introduced to delegate memory management directly to the host environment. WasmGC extends the Wasm memory model by adding struct and array heap types, thereby providing native support for non-linear memory allocation [21]. Because each WasmGC object has a fixed type and structure, host VMs can generate highly efficient access code without the risk of dynamic deoptimizations. This allows higher-level managed languages to bypass bundling their own garbage collectors, instead interfacing efficiently with the host VM's built-in garbage collector.

The architectural impact on artifact density is significant. By eliminating the need to bundle memory management code, WasmGC binaries for managed languages can achieve dramatically smaller footprints. In early benchmarks, WasmGC binaries for languages like Java were proven to be considerably smaller than equivalent modules compiled from C or Rust, because C and Rust must still bundle manual memory management routines like `malloc` and `free` inside the binary [19]. This advancement firmly positions Wasm as a viable, high-density target for the entire spectrum of modern programming languages.

In practice, however, the maturity of compilation toolchains varies sharply across the languages that nominally compile to Wasm. Although Wasm serves as a compilation target for more than forty languages [15], only a subset of them currently produce production-grade server-side artifacts. Rust occupies the most mature tier: its `wasm32-wasip1` and `wasm32-wasip2` targets are first-class compiler targets, and the Spin and Wasmtime SDKs are themselves written in Rust. Go reaches the same tier through two distinct paths: standard Go 1.21+ ships an experimental `wasip1` port using `GOOS=wasip1, GOARCH=wasm` [22], and TinyGo provides a minimal-runtime alternative that compiles to smaller binaries at the cost of partial standard-library coverage [23]. Managed languages including C#, Python, Kotlin, and Ruby remain in an emerging tier, where toolchains exist but production-grade WASI P2 component support is still arriving; WasmGC is the standardisation work that unlocks efficient compilation for these languages by delegating memory management to the host runtime, as described above [21]. Liu et al. [8] confirm that the maturity gap between Rust and managed languages remains the dominant factor in whether a Wasm container outperforms the equivalent Docker container in practice.

2.3.5 The Server-Side Evolution: WASI

This subsection introduces WebAssembly System Interface (WASI), the system interface that enables Wasm to be used as a server-side runtime—and without which the experiments in this thesis would not be possible.

While Wasm successfully delivered near-native performance for computationally heavy browser applications, its portability, structural memory isolation model, and minimal footprint rapidly garnered interest for backend deployment. Solomon Hykes, the founder of Docker, once famously stated:

> If WASM+WASI existed in 2008, we wouldn't have needed to create Docker. That's how important it is. WebAssembly on the server is the future of computing. [24]

Because Wasm is entirely isolated by default, a bare Wasm module running on a server cannot open a network socket, read a file, or query the system clock. In 2019, the WebAssembly System Interface

was introduced to solve this. WASI defines a standardized API specification that provides Wasm modules with secure, granular, capability-based access to the host operating system [5]. By acting as a secure POSIX-like layer, WASI transforms Wasm from a sandboxed browser calculator into a fully functional server-side technology.

2.3.6 Capability-Based Security Model

This subsection details WASI's deny-by-default security model, which contrasts fundamentally with Linux container permissions and is directly relevant to the security trade-offs analysed in Chapter 3.

WASI implements a capability-based security model that diverges significantly from traditional Linux permissions [5]. A Linux container limits an already-running OS environment. WASI, however, enforces a strict "deny-by-default" posture embodying the Principle of Least Authority (POLA).

If a Wasm module needs to access a configuration file, it cannot invoke a standard `open()` system call for an arbitrary path (e.g., `/etc/config`). Instead, the host runtime must explicitly grant the module a "pre-opened" file descriptor at startup. The module is completely blind to the existence of the host filesystem outside the specific directory tree explicitly mapped and passed to it by the runtime. Figure 2.6 contrasts this capability-based model with the traditional Linux permission model.

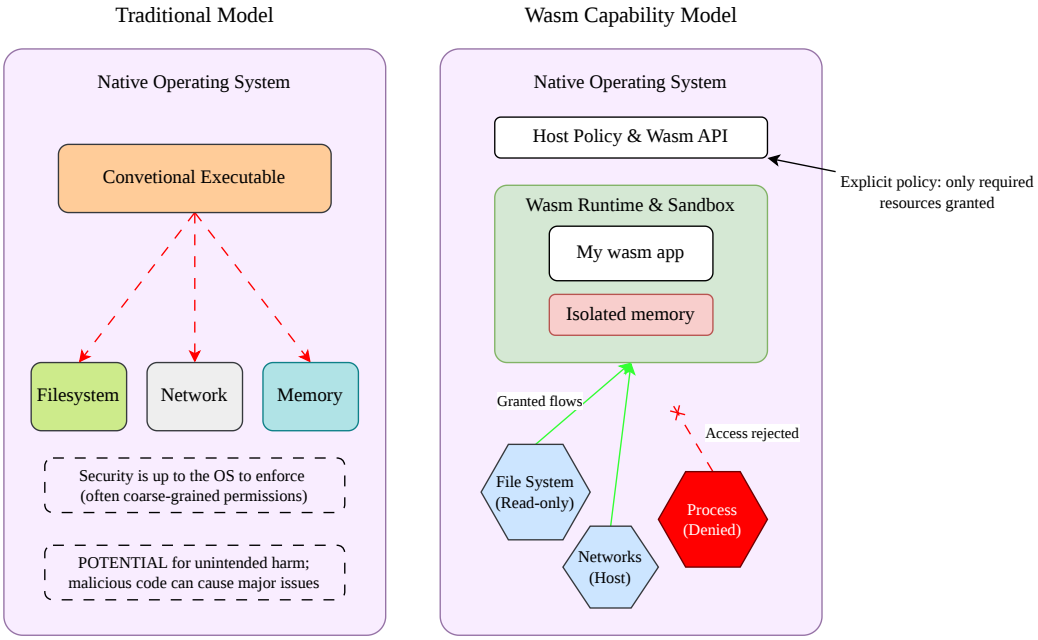


Figure 2.6 – Comparative Security Models: Traditional Linux Permissions vs. Wasm Capability-Based Access (WASI)

2.3.7 Architectural Evolution: From POSIX to the Component Model

This subsection traces the evolution from WASI P1—the version used in this thesis—to the WebAssembly Component Model (WASI P2), explaining both why Preview 1 was chosen for the current experiments and what the transition to Preview 2 will resolve.

The initial iteration of WASI (Preview 1) was heavily modeled after standard POSIX interfaces, focusing on file descriptors, environment variables, and standard I/O streams [5]. While effective for

simple command-line tools and basic microservices, this POSIX-centric approach introduced friction when handling complex, multi-language cloud-native interactions [8].

2.3.7.1 The Interoperability Problem in WASI P1

WASI P1's memory model is a significant limitation for composing services written in multiple languages. When two Wasm modules need to share data, they must agree on a shared linear memory layout—effectively a manual, type-unsafe ABI contract. This is analogous to passing raw C structs across shared memory boundaries: fragile, language-specific, and a source of subtle bugs as either module evolves [25].

This friction is especially limiting for the microservice use case. A microservice architecture ideally allows individual services to be implemented in the most suitable language. WASI P1 forces all cross-module communication through either network HTTP calls—which carry full serialization and network overhead—or through the brittle shared-memory ABI described above.

2.3.7.2 The Component Model: WIT and Typed Interfaces

To address this, the Wasm ecosystem is currently transitioning to the WebAssembly Component Model (WASI P2 and Preview 3) [25]. WebAssembly Interface Types (WIT) is a typed, language-agnostic interface definition language that allows developers to express function signatures and data types at a high level, independently of any specific language's memory layout. The Component Model shifts WASI from a low-level POSIX API to a high-level, language-agnostic interface system utilizing WIT.

A Wasm Component is a self-describing unit that declares its capabilities through WIT interfaces. When two components are composed—either statically at build time or dynamically at runtime—the Wasm runtime validates that interface contracts are satisfied and generates the necessary glue code to safely pass complex types (strings, records, variants, lists) across component boundaries without sharing linear memory. This is conceptually similar to gRPC (gRPC)/Protobuf for inter-service communication, but with zero network overhead: the call happens in-process between two sandboxed modules. This advancement allows discrete Wasm components—potentially written in entirely different source languages—to natively interoperate, laying the foundation for highly modular, secure, and dynamically composable microservice architectures.

2.3.7.3 Nano-Process Composition and Zero-Network Microservices

The Component Model enables a "nano-process" architecture where fine-grained Wasm components—each encapsulating a single responsibility—are composed into a larger application [25]. Unlike traditional microservices that communicate over HTTP, composed Wasm components communicate via in-process WIT function calls, eliminating serialization and network round-trip overhead entirely.

This architectural model is particularly compelling for the microservice domain: it retains the isolation properties of separate services (each component has its own linear memory, capabilities, and sandbox) while recovering the performance characteristics of in-process function calls. Whether the resulting latency and throughput advantages materialize in production deployments is an open research question beyond the scope of this thesis.

2.3.7.4 WASI P2 Status and Relevance to This Thesis

WASI P2 was ratified by the Bytecode Alliance in February 2024 [26]. Its stable interface worlds include `wasi:cli`, `wasi:filesystem`, `wasi:http`, `wasi:sockets`, and `wasi:io/poll` (the async I/O poll interface). The `wasi:http` world in particular provides a fully asynchronous HTTP handler interface that directly resolves the single-threaded bottleneck documented in this thesis: async HTTP frameworks targeting `wasi:http` can interleave requests without blocking, enabling multi-request concurrency on a single-threaded event loop.

Table 2.1 summarises the key differences between the two versions.

Table 2.1 – Key differences between WASI P1 and P2

Aspect	WASI P1	WASI P2 (Component Model)
Interface style	POSIX-like (file descriptors)	WIT typed interfaces
HTTP server	No standard; runtime-specific extensions	`wasi:http` async handler
Concurrency	Single-threaded I/O	Event-driven async I/O
Cross-language	Brittle shared-memory ABI	WIT composability, type-safe
Multi-module composition	Network calls or raw memory	In-process WIT function calls
Production readiness	Widely deployed for early server-side Wasm	Ratified Feb 2024; used in this thesis (SpinKube)

The primary benchmark variants use WASI P2 via SpinKube/Wasmtime (Cranelift JIT). The WASI P2 Component Model's `wasi:http/incoming-handler` interface provides a standards-compliant HTTP handler model without the proprietary socket workarounds historically required under WASI P1. The full 4-variant primary matrix is described in Chapter 4 and the runtime selection rationale is detailed in §2.3.8.

Looking beyond WASI P2, the specification continues to evolve along a trajectory directly relevant to the concurrency limits documented in this thesis. WASI P3 (in development at the time of writing) targets native async as a first-class language-level construct rather than the polling primitive bolted onto WASI P2. The practical implication for microservices is that the single-threaded request-handler bottleneck observed under WASI P1 dissolves in stages: WASI P2's `wasi:http/incoming-handler` already permits interleaved request processing on a single-threaded event loop, and WASI P3 will enable true compositional concurrency across components. The thesis benchmarks therefore measure a specific point on this trajectory — a WASI P2 runtime executing a single-handler component without WASI P3's compositional async — rather than the asymptotic upper bound of the Wasm concurrency model.

2.3.8 WebAssembly Runtimes: A Comparative Overview

This subsection surveys the major server-side Wasm runtimes and justifies the selection of SpinKube/Wasmtime for the experiments in this thesis. Runtime selection is a non-trivial decision: each runtime makes different trade-offs between WASI compliance, K8s integration, compilation strategy, and language support.

The server-side Wasm ecosystem includes several runtimes with distinct design philosophies, performance characteristics, and K8s integration capabilities. Table 2.2 provides a comparative overview of the major runtimes evaluated before selecting SpinKube (Spin/Wasmtime) for this thesis.

Wasmtime [16] is the Bytecode Alliance's reference runtime, backed by the Cranelift code generator. It has the most complete WASI P2 support of any runtime and is the foundation for Fastly Compute. Its `runwasi` containerd shim enables K8s `RuntimeClass` integration.

Table 2.2 – Comparative overview of major Wasm runtimes considered for this thesis

Runtime	Backend	WASI P1	WASI P2	K8s Shim	Primary Use Case	Role in Thesis
Spin/SpinKube	Cranelift	—	✓	containerd-shim-spin	HTTP handler P2	**Selected (Primary)**
Wasmtime	Cranelift	✓	✓	runwasi	Reference, server-side	TinyGo P1 gap
Wasmer	Cranelift/LLVM	✓	Partial	Yes	Edge, embedded	Proprietary (WASIX)
WAMR	AOT/JIT/Interp	✓	Partial	—	Embedded, IoT	No K8s shim
wazero	Go native	✓	—	—	Go embedding	Library only
WasmCloud	Cranelift	—	✓	— (platform)	Actor/NATS distributed	NATS overhead

Wasmer is a commercial runtime targeting edge and embedded deployments. It supports multiple compiler backends (Cranelift, LLVM, Singlepass) and has a packaging ecosystem (`wasmer.io` registry). Its WASI P2 support is partial and evolving.

WAMR (WebAssembly Micro Runtime) is developed by Intel and targets resource-constrained embedded and IoT devices. It supports AOT, JIT, and interpreter execution modes. Its primary focus is not cloud-native K8s deployments.

wazero is a pure-Go Wasm runtime with zero CGO dependencies. It is ideal for embedding Wasm inside Go applications but does not have a K8s containerd shim.

2.3.8.1 Why SpinKube Was Chosen

The runtime selection process evaluated all runtimes in Table 2.2 against five criteria derived from the thesis experimental requirements:

1. **Standardised WASI P2 HTTP interface:** SpinKube implements `wasi:http/incoming-handler` from the WASI P2 Component Model. The component exports a single handler function; no custom TCP accept loop or socket plumbing is required. Note: TinyGo's `-target=wasip2` cannot be used (it hardwires `wasi:cli/command`); the Spin Go SDK (`github.com/spinframework/spin-go-sdk/v2` `-target=wasip1`) provides `spinhttp.Handle()` instead.

2. **Wasmtime/Cranelift backend:** Spin embeds Wasmtime, the Bytecode Alliance reference implementation with the most complete WASI P2 Component Model support.

3. **CNCF governance:** SpinKube was accepted as a CNCF Sandbox project in January 2025, providing open-source governance and long-term availability suitable for academic reproducibility.

4. **First-class language support:** The Spin SDK for Rust (`spin-sdk = "5.2.0"`) provides a `#[http_component]` macro generating a standards-compliant WASI P2 component. The Spin SDK for Go (`github.com/spinframework/spin-go-sdk/v2`) provides `spinhttp.Handle()` with standard `net/http` types—identical handler signatures to the Docker variants.

5. **Direct HTTP-in/HTTP-out model:** Unlike , which routes every request through a NATS message broker (adding a network hop that contaminates latency measurements), SpinKube's request-handler model is structurally closest to a Docker HTTP microservice, making it the scientifically fairest WASI P2 comparison point.

2.3.9 WebAssembly in Kubernetes

This subsection describes how Wasm workloads are integrated into K8s clusters, culminating in the `RuntimeClass`-based mechanism used in this thesis. This integration layer is what makes the side-by-side comparison of Docker and Wasm variants in a single cluster possible.

2.3.9.1 Evolution: From Krustlet to runwasi

The integration of Wasm into K8s has evolved through several distinct approaches over the past five years.

Krustlet (2019–2022) was the first attempt to run Wasm workloads on K8s [27]. Rather than plugging into the existing CRI layer, Krustlet implemented the K8s Kubelet API directly, presenting itself to the control plane as a node that could schedule Wasm pods. While it proved the feasibility of Wasm-native scheduling, Krustlet's approach bypassed the standard CRI and required a separate kubelet process per node. The project entered maintenance mode in 2022 due to the difficulty of faking container-runtime semantics inside a kubelet implementation, with maintainers moving on to alternative approaches based on containerd shims [28].

runwasi [29] (Bytecode Alliance, 2022–present) is the CRI-compliant successor. It implements the `containerd` shim v2 protocol, allowing containerd to delegate Wasm pod execution to any registered Wasm runtime. From the K8s control plane's perspective, Wasm pods are indistinguishable from standard OCI pods; the routing occurs transparently inside containerd based on the pod's `runtimeClassName`. This design allows Wasm and Docker workloads to coexist on the same node without any K8s API changes.

2.3.9.2 Modern Wasm Operators for Kubernetes

Several higher-level operators have been developed to simplify the operational experience of running Wasm workloads on K8s.

SpinKube [30] ships a dedicated containerd shim (`containerd-shim-spin-v2`) that embeds the Spin runtime, which in turn embeds Wasmtime/Cranelift. It adds a `SpinApp` CRD that manages the lifecycle of Spin-based Wasm applications. SpinKube was accepted as a CNCF Sandbox project in January 2025. It is the selected Wasm framework for the primary benchmark variants in this thesis (`spin/rust`, `spin/tinygo`).

WasmCloud Operator bridges K8s scheduling with the WasmCloud lattice, routing pod scheduling decisions to WasmCloud's NATS-based actor runtime. WasmCloud was evaluated and rejected for the WASI P2 variants: its NATS broker adds a network hop that contaminates latency measurements, making a direct performance comparison against Docker HTTP microservices scientifically unsound.

KWasm is a K8s operator that automates the installation of Wasm runtime shims on cluster nodes via a DaemonSet, replacing the manual cloud-init bootstrap approach. KWasm is a meta-tool (shim installer), not a runtime; it was not used in this thesis.

For the primary SpinKube variants (wasm-rust on WASI P2, wasm-tinygo on WASI P1 via the Spin Go SDK's bridge), the `SpinApp` CRD (managed by the SpinOperator) is used, as it is the canonical deployment mechanism for SpinKube applications.

2.3.9.3 RuntimeClass: The Bridge Between Paradigms

The K8s `RuntimeClass` API is the key mechanism that enables Wasm and Docker workloads to coexist in the same cluster. A `RuntimeClass` object maps a handler name to a registered containerd runtime plugin. When a pod specifies `spec.runtimeClassName`, the kubelet instructs containerd to use the corresponding handler rather than the default `runc` path.

This thesis registers the following `RuntimeClass` objects:

- `wasmtime-spin` — maps to `containerd-shim-spin-v2` v0.17.0 (SpinKube), which embeds Spin and Wasmtime/Cranelift. Used by the primary SpinKube variants — `spin/rust` (WASI P2 component) and `spin/tinygo` (WASI P1 via the Spin Go SDK's bridge) — via `SpinApp` CRD resources managed by the SpinOperator. This is the Wasm `RuntimeClass` used in this thesis.

The complete K8s configuration for all four primary variants is detailed in Chapter 5.

2.3.10 Wasm vs. Traditional Containers

Applying Wasm and WASI to the server yields a deployment artifact that sits functionally between a traditional VM and an OCI container, offering unique operational characteristics:

1. **Cold Start and Footprint:** Traditional OCI containers bundle an OS userland, frequently resulting in image sizes exceeding 100 MB and startup times measured in seconds. A Wasm module contains only the compiled application logic. Wasm binaries are typically just a few megabytes and can be instantiated by a host runtime in microseconds. Long et al. [31] demonstrated that Wasm runtimes can perform at least 10× faster than Docker from a cold start for short-running workloads. Hall and Ramachandran further verified that Wasm's improved cold start times lead to up to 90% faster response times for basic serverless tasks [32].

2. **Security and Density:** Wasm utilizes SFI. If a Wasm module is compromised, the attacker only controls the isolated linear memory sandbox. Because Wasm modules do not require full OS process overhead, the minimal memory footprint allows K8s nodes to achieve exponentially higher workload density.

3. **Performance Trade-offs:** Despite its advantages in density and initialization, Wasm presents concurrency constraints in its WASI P1 incarnation. Wasm itself has a ratified Threads proposal [33] supported across all major browsers and runtimes, but WASI P1 does not expose thread-spawning primitives to server-side Wasm modules, leaving them constrained to single-threaded execution. Sondell [7] demonstrated that in high-concurrency HTTP server scenarios, a Docker container can process requests significantly faster because it can scale natively across multiple CPU cores, whereas a single Wasm instance operating under WASI P1 remains bottlenecked by sequential request processing. Furthermore, Spies and Mock found that single-threaded Wasm applications perform approximately 14% slower than native x86_64 binaries due to sandbox bounds-checking overhead [34].

By utilizing WASI shims (such as `runwasi` [29]), K8s can now orchestrate these high-density Wasm workloads directly alongside traditional Docker containers, establishing the precise technological foundation for the comparative analysis executed in this research.

Chapter 3

Related Work

This chapter surveys the academic literature and industry evidence most directly relevant to the research questions of this thesis. Section 3.1 reviews quantitative performance studies comparing server-side Wasm and container workloads, security analyses contrasting Wasm's isolation model with Docker's kernel-sharing model, and compiler strategy trade-offs identified in prior work. Section 3.2 surveys the current commercial deployment landscape of server-side Wasm across edge computing providers, application frameworks, and CNCF-governed Kubernetes integrations, together with the production-scale evidence and ecosystem-maturity signals that contextualise this thesis's benchmark variants. Technical background on the Wasm Component Model, available runtimes, and the K8s integration landscape is provided in Chapter 2.

3.1 Academic Research and Benchmarks

The following section surveys peer-reviewed and academic work that measures or analyses the performance, security, and ecosystem properties of server-side Wasm relative to container-based alternatives. These studies provide the empirical baseline against which the results of this thesis are positioned and contextualised. The section is structured as follows: Section 3.1.1 covers quantitative performance benchmarks; Section 3.1.2 examines security analyses; Section 3.1.3 discusses compiler strategy trade-offs documented in prior work; Section 3.1.4 summarises recent surveys and cross-paradigm studies that synthesise and extend these threads.

3.1.1 Performance Studies

Several academic works have investigated the performance characteristics of server-side Wasm and directly inform the research questions of this thesis. The studies below are grouped by the performance dimension they address: native-binary overhead, cold-start and density advantages, and throughput under concurrent load.

Native-binary overhead. Jangda et al. [35] performed a systematic analysis of Wasm performance relative to native x86_64 code across a broad set of benchmarks from the PolyBenchC suite. They found that Wasm is on average 1.5× slower than native code, with the overhead attributable to SFI bounds-checking, function pointer masking, and the inability of the Wasm ABI to exploit all available hardware registers. Separately, Spies and Mock [34] evaluated Wasm compiled with `wasm32-wasi` in non-web environments and found that single-threaded, CPU-bound workloads run approximately 14%

slower than equivalent native x86_64 binaries [34]. This overhead stems from the bounds-checking and indirect-call guards enforced at runtime by SFI. Their study establishes an important baseline: the Wasm runtime is not free of overhead even for compute-intensive tasks, though the gap shrinks considerably with AOT compilation.

Cold-start latency and image density. Long et al. [31] conducted a direct performance comparison between Wasm and containerized workloads, demonstrating that Wasm runtimes achieve at least a 10× improvement in cold-start latency over Docker for short-running workloads. This advantage is most pronounced in Function as a Service scenarios where containers are created and destroyed frequently. Hall and Ramachandran [32] extended this finding to the serverless domain, showing that the Wasm cold-start advantage translates into up to 90% faster end-to-end response times for event-driven serverless functions [32]. Importantly, both of these cold-start advantages are primarily attributable to Wasm's dramatically smaller deployment artifacts rather than to fundamental differences in JIT compilation speed alone.

Concurrent throughput. The most directly comparable prior work to this thesis is the master's thesis by Sondell [7] at KTH Royal Institute of Technology. Sondell benchmarked Wasm containers against Docker containers for serverless HTTP workloads and found a significant throughput collapse under concurrent load for the Wasm variant. The root cause is identical to the finding documented in this thesis: WASI P1 does not expose thread-spawning primitives to modules, leaving the HTTP server constrained to processing one request at a time while Docker containers leverage native multi-core concurrency via goroutines or thread pools. Sondell's work uses a different runtime configuration and workload type; this thesis extends that line of inquiry with four implementation variants (two language families, two runtime targets), a dedicated CPU-bound compute benchmark, and a structured developer experience assessment, providing complementary empirical coverage of the Wasm/K8s deployment space.

The WASI P2 transition partially mitigates this bottleneck through two complementary mechanisms. First, `wasi:http/incoming-handler` together with the `wasi:io/poll` interface permit a single-threaded event loop to interleave multiple in-flight requests at I/O suspension points rather than serialising them [25, 26]. Second, runtimes such as Fermyon Spin can be horizontally scaled at the pod level (raising the SpinApp's `spec.replicas`) so that concurrent requests are dispatched across multiple independently scheduled component instances on multi-core hosts, even though each instance remains internally single-threaded. True intra-component multithreading is not part of WASI P2; native async at the language level is targeted for WASI P3 (in development). The throughput results reported in Chapter 6 reflect this precise trade-off: Wasm variants close most of the gap to Docker once the replica pool is allowed to dispatch concurrently, but a single component instance running a single handler remains slower than a multi-threaded Docker baseline at the same request rate.

3.1.2 Security Comparisons

The security architecture of Wasm/WASI has been studied in the context of multi-tenant cloud workloads, with analyses focusing on both the structural advantages of Wasm's isolation model and its residual attack surface.

Container security risks. Combe, Martin, and Di Pietro [36] surveyed the security properties of Docker and found that the shared-kernel model is the primary structural risk in multi-tenant deployments. If a container escape vulnerability is exploited, the attacker gains code execution at the host kernel level, potentially compromising every container on the node. Shillaker and Pietzuch

similarly documented the kernel-sharing threat model in the context of serverless isolation [3]: a single compromised container can expose the entire host to privilege escalation. Wasm, by contrast, enforces SFI at the runtime level: the module can only read and write within its allocated linear memory region, and any out-of-bounds access results in a deterministic, immediate trap. A module compromise therefore yields only control of that module's sandboxed linear memory, not host-level execution.

Wasm binary security. Lehmann, Kinder, and Pradel [37] analysed the binary security properties of Wasm modules and found that, while the linear memory model successfully prevents classical memory-safety attacks (buffer overflows, use-after-free), it introduces a new class of confused-deputy vulnerabilities at the Wasm-host interface boundary. Specifically, if a host import (a function provided by the runtime to the Wasm module) is overly permissive, an attacker with control over the module can abuse it to perform operations outside the intended capability scope. Fine-grained network capability restriction—the theoretical security advantage of WASI's deny-by-default model—depends on how conservatively host imports are defined in practice.

WASI extends the isolation model with a POLA-based capability model: network sockets, files, and environment variables are only accessible if the host runtime explicitly grants them at startup. This "deny-by-default" posture is structurally more restrictive than Linux's permission model, though as the findings above illustrate, its practical security benefit depends on how conservatively host imports are defined.

3.1.3 JIT vs. AOT Compilation

Wasm runtimes offer two primary compilation strategies that directly affect the cold-start vs. throughput trade-off, and this choice has been studied in the context of both browser and server-side deployments.

JIT compilation—the default mode in the Wasmtime/Cranelift runtime used in this thesis—translates the `.wasm` binary to native machine code at module load time. This preserves binary portability across ISAs and keeps the deployment artifact small, at the cost of a one-time JIT overhead on first pod startup. AOT compilation pre-translates the `.wasm` binary to a platform-specific native shared object before deployment, eliminating the JIT startup penalty entirely but tying the artifact to a specific architecture. Spies and Mock [34] showed that the performance gap between Wasm and native code narrows considerably under AOT compilation, suggesting that JIT overhead is a significant but avoidable contributor to the 14% slowdown they observed.

This thesis uses the JIT path for both Wasm variants to maintain binary portability and reproducibility across potential future cluster migrations. This is acknowledged as a conservative choice that may slightly disadvantage the Wasm variants in the compute benchmark; AOT compilation is left as a direction for future optimisation work.

3.1.4 Recent Surveys and Cross-Paradigm Studies

Beyond the foundational performance, security, and compiler-strategy studies surveyed above, several recent peer-reviewed works synthesise and extend the empirical basis for evaluating server-side Wasm. Zhang et al. [38] catalogue 98 papers on Wasm runtimes and propose a two-axis taxonomy that separates "internal" runtime research (design, testing, security analysis) from "external" application research (IoT, edge, cloud, blockchain), providing the most comprehensive map of the Wasm research landscape to date. Kakati and Brorsson [39] extend cross-architecture evaluation of Wasmtime and WAMR across x86_64, ARM64, and RISC-V, providing the first cross-architecture performance baseline for the cloud-edge continuum and confirming that Wasm's portability advantage is not theoretical but

empirically realisable on heterogeneous hardware. Wiegratz [40] directly investigates Wasm versus Linux containers under K8s for untrusted-code execution, finding that Wasm reduces attack surface meaningfully but introduces measurable startup overhead — a result that complements the security comparisons of Section 3.1.2 with a K8s-specific perspective. Kjorveziroski and Filiposka [9] benchmark Wasm against Docker across thirteen serverless functions, reporting that Wasm runtimes lead on ten of the thirteen with Wasmtime emerging as the fastest — a finding that anticipates the scaling-mode results of this thesis. Kjorveziroski and Filiposka [41] additionally describe a K8s operator extension for orchestrating native Wasm modules alongside OCI pods, foreshadowing the `RuntimeClass`-mediated coexistence used in this thesis. Furthermore, Liu et al. [8] present a measurement study of Wasm-based container runtimes and surface an important nuance: Wasm containers do not unconditionally outperform Docker containers, with the maturity gap of the language toolchain often dominating runtime-level differences.

3.2 Industry Adoption and Production Deployments of Server-Side WebAssembly

This section maps the current commercial deployment landscape of server-side Wasm and the production-scale evidence and ecosystem-governance signals that indicate the technology's maturity. Understanding where Wasm has already been deployed at scale provides essential context for interpreting the benchmark results in Chapter 6 and for calibrating expectations relative to the experimental deployment studied in this thesis. The first large-scale production deployments occurred at edge computing providers, where cold-start density and instantiation latency are primary commercial constraints, and the same momentum has since extended into developer frameworks and CNCF-governed Kubernetes integrations.

3.2.1 Cloudflare Workers

Cloudflare Workers [42] runs Wasm inside V8 Isolates—not a WASI runtime—by embedding the browser's JavaScript engine at the network edge. Each Worker invocation receives a fresh V8 Isolate with sub-millisecond startup across a network spanning 310+ cities in 120+ countries, making it the largest publicly documented deployment of server-side Wasm to date. Cloudflare's 2025 "Shard and Conquer" architecture further reduced cold starts to the point where 99.99% of requests are served warm, effectively eliminating cold-start penalty as a user-visible phenomenon at their scale of operation.

The relevance to this thesis is two-fold. First, the platform demonstrates that Wasm's density advantage—many isolated tenants per host, with rapid instantiation—is not merely theoretical but commercially proven at global scale. Second, the isolation mechanism (V8 Isolates rather than a WASI runtime) highlights that production-grade Wasm deployments can diverge significantly in their security and capability models; the WASI-based approach evaluated in this thesis represents a different, K8s-native deployment pattern.

3.2.2 Fastly and the Convergence to Wasmtime

Fastly Compute [43] built its edge computing platform on Lucet, an AOT-compiling Wasm runtime developed internally and open-sourced in 2019 as the then state of the art for serverless Wasm performance. Rather than replacing Lucet outright, Fastly subsequently migrated its key innovations—

the pooling instance allocator and the AOT compilation pipeline—directly into Wasmtime, the Bytecode Alliance's reference runtime [44], and Fastly Compute now runs on Wasmtime in production.

This convergence is significant from an ecosystem perspective: even a well-funded, production-grade custom runtime found it strategically preferable to merge into the community reference implementation rather than maintain a divergent fork. The result is a stronger shared base for all Wasm runtimes and evidence that the Bytecode Alliance governance model produces long-term ecosystem stability—a factor relevant to any organisation evaluating a migration to Wasm-based infrastructure.

3.2.3 Application Frameworks and Platforms

Fermyon Spin [45] is a developer-centric framework for building serverless Wasm microservices. Spin uses a request-scoped execution model based on WASI P2's `wasi:http/incoming-handler`: Spin owns the TCP listener, and the component exports a handler function invoked once per request. Spin embeds Wasmtime/Cranelift internally as its execution engine. SpinKube extends Spin to Kubernetes via the `containerd-shim-spin-v2` containerd shim and a `SpinApp` CRD, making Spin the runtime of choice for this thesis; the detailed selection rationale is presented in Section 2.3.8.

WasmCloud [46] is an actor-model distributed application platform. Services are implemented as location-transparent actors that communicate over a NATS message bus; the runtime injects capability providers for networking, databases, and other host resources at launch time without baking them into the actor binary. WasmCloud targets the WASI P2 Component Model and is a CNCF-sponsored project. Its distributed-systems orientation makes it a strong candidate for the Volta microservice architecture planned as future work in this thesis.

3.2.4 CNCF Ecosystem Maturity

The governance investment by the CNCF in server-side Wasm projects provides a quantifiable maturity signal. SpinKube, the K8s operator for running Fermyon Spin applications natively, entered the CNCF Sandbox on 21 January 2025 [30]—the same maturity track used by Kubernetes, containerd, and Prometheus before they graduated. runwasi, the Rust library for writing containerd shims for Wasm workloads, is actively maintained under the containerd organisation and is used in production by both K3s [47] and Microsoft's Azure Kubernetes Service to execute Wasm workloads via `RuntimeClass` [29]. The CNCF TAG-Runtime Wasm Working Group [48] was chartered to define best practices for Wasm as a first-class cloud-native workload type alongside OCI containers.

Collectively, these governance milestones indicate that server-side Wasm has crossed from a research curiosity to an infrastructure category with active stewardship by the same foundation that governs the broader cloud-native ecosystem. This is directly relevant to this thesis: SpinKube is the primary CNCF-affiliated runtime component used in the experimental cluster, orchestrated by upstream Kubernetes (via `kubeadm`) through the standard `RuntimeClass` API.

Chapter 4

Research Methodology

This chapter describes the research strategy employed to compare Wasm and Docker-based microservice deployments, justifies the selection of the four implementation variants, defines the benchmark workloads, and specifies the five measurement metrics used throughout the evaluation.

4.1 Research Strategy

This thesis employs a *mixed-methods* approach combining a quantitative controlled experiment as the primary method with a qualitative developer experience assessment as the secondary method.

The primary quantitative experiment follows a 2×2 variant matrix: runtime (Docker/`runc` vs. Wasm/SpinKube) crossed with language family (Rust vs. Go-family). This produces four primary implementation variants deployed to an isolated, reproducible cloud environment and subjected to identical load testing and startup-time measurements. All variants implement the same HTTP API and the same core algorithm, ensuring that any performance difference is attributable to the execution environment rather than to differences in application logic. The benchmark source code, k6 load-generator scripts, and raw result data are openly available in the `thesis-experiments` repository [49].

The Wasm variants use the standardised `wasi:http/incoming-handler` Component Model interface (WASI P2) via Fermyon Spin and Wasmtime/Cranelift. This design choice reflects the current production-ready trajectory of Wasm on K8s: the WASI P2 Component Model provides a standards-compliant HTTP handler interface, replacing the proprietary socket workarounds historically required under WASI P1.

An important design choice is that this study does *not* attempt to isolate the pure runtime overhead by holding the language constant across all four variants. Instead, it deliberately compares the two dominant paradigms in their *native* technological contexts: Docker with Go (the industry standard), Docker with Rust (a deconfounding baseline), and Wasm with Rust and TinyGo (the Wasm-native ecosystem). This design reflects the real-world decision an engineering team faces when evaluating a migration from Docker to Wasm, while the Rust-on-Docker baseline enables partial deconfounding of the language variable (see Section 4.2).

4.2 Selection of Technologies

4.2.1 Docker/Golang: The Industry Baseline

The Docker/Golang variant serves as the primary baseline against which all other variants are measured. Go is the de facto language for cloud-native infrastructure tooling: K8s [2], Docker [1], Terraform [50], Prometheus [51], and Grafana are all implemented in Go. Its goroutine-based concurrency model and the `net/http` standard library make it the idiomatic choice for HTTP microservices. Among production-grade garbage-collected languages, Go offers competitive raw throughput while retaining ergonomic memory safety, making it a strong baseline for evaluating whether Wasm's elimination of bundled garbage collection overhead provides a meaningful performance advantage.

4.2.2 Docker/Rust: The Deconfounding Baseline

A second Docker-based baseline using Rust is included specifically to enable a language-controlled comparison. Because both docker-rust and wasm-rust use the same language, the performance delta between these two variants isolates (approximately) the overhead introduced by the SpinKube/Wasmtime runtime, SFI bounds-checking, and the single-instance dispatch constraint imposed by the `SpinApp spec.replicas: 1` limited-mode baseline (each WASI P1/P2 component is architecturally single-threaded), independent of language-level differences such as memory management strategy.

Rust's axum/Tokio async stack provides full multi-threaded, asynchronous I/O—the maximum performance achievable on the Docker/`runc` path—making the docker-rust variant also a useful upper-bound reference for the Wasm variants.

4.2.3 Wasm/Rust: The Primary Experimental Variant

Wasm supports more than 40 programming languages as compilation targets [15]. Rust was selected as the primary Wasm experimental language for three reasons. First, Rust provides first-class, officially supported `wasm32-wasip1` and `wasm32-wasip2` targets maintained by the Rust core team; `wasm32-wasip2` has been Tier 2 stable since Rust 1.82 [52]. Second, Rust produces the smallest Wasm binaries among systems languages because it has no garbage collector, no language runtime, and no OS abstraction layer bundled into the binary. Third, the Spin SDK for Rust (`spin-sdk = "5.2.0"`) provides a first-class `#[http_component]` macro that generates a standards-compliant WASI P2 component with no custom networking code.

The wasm-rust variant uses `cargo component build --release` (via `cargo-component`) to compile to a WASI P2 component. The resulting `.wasm` binary is published as a Spin OCI artifact via `spin registry push`. Spin's Wasmtime/Cranelift host handles the HTTP listener, request parsing, and response dispatch—the component itself exports only a single handler function, eliminating any need for manual TCP socket management.

4.2.4 Wasm/TinyGo: The Secondary Experimental Variant

TinyGo [23] is a Go-compatible compiler optimised for embedded systems and Wasm targets. It was selected to enable a partial Go-to-Go comparison with the docker-golang baseline, testing whether the concurrency model difference between Docker's goroutine scheduler and Wasm's request-handler model produces measurable performance differences when the language is held constant.

Standard Go supports GOOS=wasip1 since Go 1.21 [22], but produces significantly larger Wasm binaries because it bundles the full Go runtime (scheduler, garbage collector, and POSIX abstraction layer). TinyGo is designed specifically for Wasm/embedded targets and produces substantially smaller binaries by using a stripped-down runtime. TinyGo 0.40.0 targeting wasip1 was pinned for this experiment.

The wasm-tinygo variant uses spinhttp.Handle() from the official Spin SDK for Go (github.com/spinframework/sp The handler function receives standard net/http.ResponseWriter and *http.Request arguments—identical to the docker-golang handler. **No custom socket code is required.** TinyGo's -target=wasip2 flag cannot be used here: it hardwires the wasi:cli/command world and cannot export wasi:http/incoming-handler The Spin Go SDK uses a layer to export fermyon:spin/inbound-http, which Spin/Wasmtime accepts as a valid HTTP trigger interface. The -target=wasip1 flag is therefore required.

4.2.5 Orchestration: Kubernetes (kubeadm) with RuntimeClass

Upstream Kubernetes v1.34, deployed via kubeadm, was chosen as the orchestration layer. kubeadm is the reference installer in the official Kubernetes documentation and the foundation that managed services such as , Google , and Azure AKS run underneath; using it removes distribution-specific confounds from the runtime comparison. The benchmark VM is sized at Hetzner Cloud ccx23 (4 dedicated AMD vCPUs, 16 RAM, 160) to give the kubeadm control plane, the kube-prometheus-stack observability stack, the SpinOperator, and the unlimited-mode scaling experiment comfortable headroom on a single node. kubeadm uses containerd 2.x as its CRI, which is necessary for the RuntimeClass/SpinKube integration; Flannel provides the pod-network and Rancher's local-path-provisioner provides the default StorageClass for the Prometheus PVC.

The Wasm RuntimeClass (wasmtime-spin) routes pods to containerd-shim-spin-v2 v0.17.0, which embeds Spin and Wasmtime/Cranelift internally to execute WASI P2 components. Docker pods use the default runc path unchanged. This hybrid configuration allows both paradigms to run side-by-side with directly comparable K8s resource management.

4.3 Benchmark Workload Design

4.3.1 Sieve of Eratosthenes as the Compute Benchmark

The first benchmark workload is a stateless HTTP service that computes prime numbers using the Sieve of Eratosthenes algorithm. The sieve was selected as the opening benchmark for several reasons. It is purely CPU-bound with no external I/O, database calls, or file system access, isolating compute performance from I/O stack differences between the variants. Its $O(N \log \log N)$ time complexity produces a predictable, parametrically controllable workload: the upper bound N can be adjusted to modulate per-request computation time precisely. The algorithm is deterministic and trivially re-implementable in any language, ensuring that all six variants compute identical results using the same algorithmic logic.

The HTTP API contract is identical across all six variants:

```
GET /prime?limit=N&no_list=1
```

The response is a JSON object:

```
{"runtime":"<variant>","limit":N,"count":K,"elapsed_us":T}
```

The `no_list=1` parameter suppresses the array of computed primes from the response body, eliminating JSON serialization overhead as a confounding variable when measuring throughput. The `elapsed_us` field records the server-side wall-clock time of the sieve computation in microseconds, providing a direct measurement of per-request algorithmic cost independent of network and queuing latency.

Table 4.1 summarizes the four primary implementation variants.

Table 4.1 – Summary of the four primary benchmark variants

Variant	Language	Runtime	Framework	I/O Model	NodePort
wasm-rust	Rust 1.94	Wasmtime/Cranelift	`spin-sdk #[http_component]`	WASI P2 handler	30081
wasm-tinygo	TinyGo 0.40.0	Wasmtime/Cranelift	`spinhttp.Handle()`	Spin HTTP (WASI P1 via CGo)	30082
docker-rust	Rust 1.94	runc	axum + Tokio	Async (multi-thread)	30083
docker-golang	Go 1.26	runc	`net/http`	Goroutines (M:N)	30084

4.3.2 Memory-Bandwidth Benchmark

The second benchmark workload is a stateless HTTP service that allocates an in-memory buffer of configurable size, fills it with deterministic bytes, computes its -256 hash, and returns the hash and elapsed time. This workload was designed to contrast with the CPU-bound prime sieve by stressing memory-allocation patterns and the runtime's ability to handle repeated large-heap operations.

Note on filesystem I/O: The original design called for writing to and reading from a temporary file. However, SpinKube's Wasmtime host exposes `wasi:filesystem` access only to explicitly declared directories in `spin.toml`; ephemeral `/tmp` writes are not available in the default SpinApp sandbox. The in-memory buffer approach provides an equivalent memory-pressure workload that is consistent across all four variants (Docker variants perform the same in-memory computation without filesystem involvement), preserving the fairness of the comparison.

The HTTP API contract is identical across all four variants:

```
GET /membw?size_kb=N&no_hash=0
```

The response is a JSON object:

```
{"runtime":"<variant>","size_kb":N,"sha256":"<hex>","elapsed_us":T}
```

The `no_hash=1` parameter suppresses the hash computation (returning an empty string), enabling measurement of raw allocation overhead independently of hashing time. Default `size_kb=64`; maximum `size_kb=10240`. The benchmark uses `size_kb=64` for load testing and `size_kb=1024` for stress testing large-allocation behaviour.

4.3.3 Planned Future Workloads

- **Volta Financial Risk Engine:** A distributed multi-service application implementing a Monte Carlo simulation for Value at Risk (VaR) calculations on trading portfolios. Volta consists of multiple communicating microservices, stress-testing Wasm's suitability for realistic, stateful, inter-service architectures of the type that dominate modern cloud deployments.

4.4 Definition of Metrics

Five metrics are measured across all variants. The first four are quantitative; the fifth is qualitative.

1. **Startup Latency (Cold and Warm Start).** Time elapsed from `kubectl scale --replicas=0` followed by `kubectl scale --replicas=1` until the service returns the first HTTP 200 response, measured by a polling script. This elapsed time includes the period required for the previous pod instance to terminate, plus any OCI image pull time (for cold starts), plus runtime initialisation. Six measurement runs are performed per variant. Run 1 constitutes the *cold start*: the node has no cached image layer, so the OCI image is pulled from the registry before the pod can start; only the single observed value is reported for cold start. Runs 2–6 constitute *warm starts*: the image is already cached on the node, so pod startup reflects only container/Wasm runtime initialisation time; mean, median, and standard deviation are computed across these five warm-start runs. All latency values are expressed in milliseconds.

2. **Memory Footprint.** Resident Set Size (RSS) is the portion of a process's memory held in physical RAM at a given point in time; it excludes memory swapped to disk and memory-mapped files that are not currently resident, making it the most practical measure of the RAM a running service occupies on a production node. RSS is reported by Prometheus [51] via the `container_memory_rss` metric, collected at two operating points: *idle* (pod running for 60 s with no incoming requests) and *peak* (maximum RSS observed during the k6 load test window). Scrape interval: 5 s. Units: megabytes.

3. **OCI Artifact Size.** The on-disk size of the OCI image as reported by `docker inspect`. This metric directly impacts image pull latency, registry storage costs, and cold-start time when images are not cached. Units: megabytes.

4. **Throughput and Latency Under Load.** Load testing is performed with k6 [53] using a ramping-VU executor. k6 models load using VU: each VU is an independent simulated client that continuously sends requests to the target service for the duration of its active phase, waiting for a response before issuing the next request; the ramp-up phase gradually increases concurrency to avoid an instantaneous spike that would distort latency measurements. The load profile ramps from 1 VU linearly to 50 VUs over 20 s, holds at 50 VUs for 60 s, then ramps down to 0 over 10 s. All requests target `GET /prime?limit=100000&no_list=1`. Primary metrics: requests per second (RPS), HTTP request failure rate (fraction of non-2xx responses), p50/p95/p99 end-to-end latency, and the custom `server_compute_us` metric extracted from the JSON response body to measure server-side algorithm time independently of network and queuing effects.

5. **Developer Experience (Qualitative).** A structured narrative assessment of the engineering effort required to build, test, and deploy each variant. Four dimensions are evaluated using a four-point friction scale (Low / Medium / High / Very High): (a) toolchain setup and installation, (b) dependency compatibility under WASI (Preview 1 and Preview 2), (c) K8s cluster integration complexity, and (d) debugging and observability capability. This assessment is presented in Chapter 6 and directly addresses RQ4.

Chapter 5

Implementation

This chapter describes the end-to-end implementation of the benchmark environment: the rationale for using a dedicated cloud VM, the infrastructure provisioning process, the design of the reference workload, all four service implementations, and the K8s deployment configuration.

5.1 Why a Dedicated Cloud Virtual Machine

The primary goal of the experimental infrastructure is measurement *isolation*: ensuring that observed performance differences between variants reflect genuine runtime and language characteristics, not environmental noise. Three alternative approaches were evaluated before settling on a dedicated Hetzner Cloud VM:

- **Local Minikube:** Runs a K8s cluster inside a VM on the developer's laptop. Background processes (browser, IDE, system updates) compete for CPU and memory, making benchmark results unreproducible across sessions. Additionally, consumer laptop CPU governors apply dynamic frequency scaling under load, introducing measurement variance.
- **k3d (K3s in Docker):** Runs a K3s cluster inside Docker containers on the local machine. This introduces multiple hypervisor/container nesting layers—host OS → Docker → K3s → containerd → Wasm runtime—that obscure the runtime overhead being measured and complicate the registration of the SpinKube containerd shim.
- **Managed Cloud K8s (EKS, GKE, AKS):** Managed K8s offerings do not support custom OCI runtimes or `RuntimeClass` with non-standard shims out of the box. Adding SpinKube (`containerd-shim-spin-v2`) support requires custom node groups or operator-managed DaemonSets, adding abstraction layers that complicate fair comparison.

A dedicated Hetzner Cloud VM provides a single-purpose environment with no competing workloads, full control over kernel configuration and containerd setup, and complete reproducibility via Infrastructure as Code (IaC) [50]; the provisioning Terraform modules and cloud-init bootstrap scripts are published as the `thesis-infra-setup` repository [54]. The Hetzner ccx-series CPU type is a dedicated-vCPU instance family; occasional noisy-neighbour effects from co-located tenants on the underlying host may still introduce outliers in absolute metric values. This is mitigated by repeating measurements across multiple runs and reporting means and standard deviations.

5.2 Infrastructure Setup

5.2.1 Hardware and Kubernetes Distribution

The benchmark cluster runs on a Hetzner Cloud ccx23 instance [55]. Hetzner Online GmbH is a German cloud and hosting provider offering cost-efficient dedicated and shared virtual machines in European data centers; the ccx23 is a dedicated-CPU instance type with four AMD EPYC vCPUs, 16 GB RAM, 160 GB NVMe SSD, running Ubuntu 24.04 LTS in the Nuremberg (nbg1) datacenter. The entire server provisioning is defined in Terraform [50]—an open-source Infrastructure as Code tool that declaratively manages cloud resources—and executed with a single `make up` command.

Upstream Kubernetes v1.34, deployed via `kubeadm`, is installed as the K8s distribution. `kubeadm` is the reference installer in the official Kubernetes documentation and is the foundation that managed services such as , Google , and Azure AKS run underneath; using it removes distribution-specific confounds from the runtime comparison. The ccx23 sizing provides ample headroom above the combined footprint of the kubeadm control plane (apiserver, etcd, controller-manager, scheduler, kube-proxy), `cert-manager`, the SpinOperator, the kube-prometheus-stack observability stack, and the benchmark workload pods. Services are exposed via NodePort directly without an ingress controller, to avoid introducing an irrelevant processing layer between the load generator and the measured services. containerd 2.x is used as the CRI—the standardized interface through which the kubelet communicates with the container runtime to manage pod lifecycles—and Flannel provides the pod-network .

5.2.2 The SpinKube Runtime Chain

Running Wasm pods alongside standard OCI pods requires configuring containerd to route Wasm workloads through a different OCI runtime. The full SpinKube chain is:

`kubelet` → `containerd` → `containerd-shim-spin-v2` → **Wasmtime/Cranelift** → `.wasm component`

The key components in this chain are:

- **containerd-shim-spin-v2 v0.17.0:** A containerd shim that implements the containerd shim v2 protocol and embeds Fermyon Spin (which embeds Wasmtime/Cranelift) internally. It does *not* use crun or runc; the shim is the direct process host for the Wasm component. Installed from the SpinKube GitHub release as a static binary at `/usr/local/bin/containerd-shim-spin-v2`.

- **containerd plugin entry:** The containerd configuration registers a `spin` runtime plugin with `runtime_type = 'io.containerd.spin.v2'`. This is always active; no pod annotations are required.

- **SpinOperator (v0.6.1):** A Kubernetes operator (CNCF Sandbox) that watches `SpinApp` CRD resources and creates managed `Deployments` with `runtimeClassName: wasmtime-spin`. cert-manager v1.16.3 is a prerequisite for the operator's admission webhook.

- **Wasmtime/Cranelift:** The JIT engine embedded inside Spin. All WASI P2 component execution uses Wasmtime with the Cranelift JIT backend.

5.2.3 Observability Stack

Observability refers to the ability to infer the internal state of a system from its external outputs—primarily metrics, logs, and traces. In this experimental context, observability is required to collect the quantitative performance metrics defined in Chapter 4 (memory, CPU, request latency, and throughput) from all six variants simultaneously and under identical conditions.

The `kube-prometheus-stack` Helm chart is deployed into the `observability` namespace, providing Prometheus [51] (NodePort 32090, 5 s scrape interval, 7-day retention, 10 Gi PVC) and Grafana (NodePort 32000). A 5-second scrape interval was chosen to capture sub-second metric resolution during the 60-second k6 load window. Node Exporter provides host-level CPU and memory metrics; the `cAdvisor` integration in containerd exposes per-container `container_memory_rss` and `container_cpu_usage_seconds_total` metrics.

5.2.4 Infrastructure as Code

The entire environment is described declaratively:

- **Terraform** provisions the Hetzner Cloud firewall (restricting SSH and K8s API access to the operator's IP) and the ccx23 VM.
- **cloud-init** installs containerd 2.x, `kubeadm/kubelet/kubectl` (v1.34), runs `kubeadm init`, installs the Flannel and `local-path-provisioner`, untaints the control-plane node, and installs `containerd-shim-spin-v2`. The complete environment, from bare cloud VM to a functioning SpinKube-enabled K8s cluster with observability, is bootstrapped with `make up && make configure && make deploy`.

5.3 Reference Workload: Sieve of Eratosthenes

The Sieve of Eratosthenes is a classical algorithm for finding all prime numbers up to a given limit N. The implementation marks composite numbers by iterating through multiples of each discovered prime, completing in $O(N \log \log N)$ time with $O(N)$ space. All six variants implement the identical algorithmic logic; the only differences are the HTTP framework and I/O model.

The HTTP API is:

```
GET /prime?limit=N&no_list=1
```

where `limit` is capped at 10,000,000 across all variants. The JSON response contains the fields `runtime`, `limit`, `count`, and `elapsed_us`. The `elapsed_us` field measures only the sieve computation, not network or serialization time. The `no_list` flag suppresses the array of prime numbers, ensuring that JSON encoding time does not confound throughput measurements.

Each variant also exposes `GET /health` returning HTTP 200, used for K8s liveness and readiness probes.

5.4 Docker/Golang Implementation

The docker-golang service uses Go 1.26 and the `net/http` standard library. The server is started with `http.ListenAndServe(":8080", mux)`, which creates one goroutine per accepted connection. Go's

M:N scheduler multiplexes goroutines onto OS threads, enabling concurrent request handling without explicit thread management. The sieve computation and JSON marshaling use only standard library packages (`encoding/json`), resulting in zero external dependencies.

The Docker image uses a two-stage build: a `golang:1.26-alpine` builder stage compiles a fully static binary with `CGO_ENABLED=0 GOOS=linux`, and a `scratch` final stage contains only the binary. The resulting image is **6.00 MB**, the largest of the Docker/WASI P1 variants due to Go's compiled-in runtime metadata (goroutine scheduler, garbage collector, standard library DWARF symbols).

5.5 Docker/Rust Implementation

The docker-rust service uses Rust 1.94 with the axum 0.8 web framework running on the Tokio async runtime. The `#[tokio::main]` macro spawns a multi-threaded executor whose thread count equals the number of available CPU cores. Axum's async handler model allows all 50 concurrent k6 VUs to be processed in parallel, with each handler awaiting the sieve computation in its own async task.

The Docker image uses a two-stage build: a `rust:1.94-slim` builder stage with `cargo build --release` (link-time optimisation enabled, `codegen-units=1`, `opt-level=3`, binary stripped), and a `scratch` final stage. The resulting image is **2.08 MB**—3× smaller than docker-golang, reflecting Rust's lack of a garbage collector and the absence of any bundled runtime metadata.

5.6 SpinKube/Rust Implementation

5.6.1 HTTP Handler via spin-sdk

The wasm-rust variant uses `spin-sdk = "5.2.0"`, the official Spin SDK for Rust WASI P2 components. The `#[http_component]` procedural macro generates the WASI P2 `wasi:http/incoming-handler` export automatically:

```
use spin_sdk::http::{IncomingRequest, Response};
use spin_sdk::http_component;

#[http_component]
fn handle(req: IncomingRequest) -> Response {
    match req.path_with_query().as_deref() {
        Some(p) if p.starts_with("/prime") => sieve_handler(&req),
        Some("/health") => health_handler(),
        _ => Response::builder().status(404).body("not found").build(),
    }
}
```

There is no TCP socket management, no accept loop, and no custom HTTP parsing. Spin's Wasmtime/Cranelift host handles all of that; the component receives a parsed `IncomingRequest` and returns a `Response`.

5.6.2 Build and Packaging

The component is built with `cargo component build --release` (from the `cargo-component` toolchain), which compiles to a WASI P2 component and runs `wasm-tools component link` to produce a self-contained `.wasm` file. The artifact is pushed to the registry as a Spin OCI artifact—not a standard Docker image—using `spin registry push docker.io/abdulaziz7225/prime-sieve-wasm-rust:latest`. Spin OCI artifacts use distinct media types (`application/vnd.fermyon.spin.manifest.v2+json`) that require `containerd-shim-spin-v2` to pull and execute correctly.

Concurrency is capped at the pod level: the limited-mode baseline runs the `SpinApp` with `spec.replicas: 1`, and each WASI P1/P2 component is architecturally single-threaded per Wasmtime instance, satisfying the resource-fairness requirement without any in-component setting. (Spin 2.x [`spin_manifest_-version = 2`] no longer accepts the legacy `max_instances` field that earlier Spin 1.x versions exposed in `spin.toml`.) The compiled `.wasm` artifact is typically under 1 MB stripped.

Source: `wasm/rust/01-prime-sieve/`

5.7 SpinKube/TinyGo Implementation

5.7.1 HTTP Handler via Spin Go SDK

The wasm-tinygo variant uses the official Spin SDK for Go (`github.com/spinframework/spin-go-sdk/v2`). The `spinhttp.Handle()` function registers the application's HTTP handler via a layer that exports `fermyon:spin/inbound-http`, the interface Spin/Wasmtime dispatches HTTP requests through. The handler receives standard `net/http.ResponseWriter` and `*http.Request` arguments:

```
func init() {
    mux := http.NewServeMux()
    mux.HandleFunc("/prime",  sieveHandler)
    mux.HandleFunc("/health", healthHandler)
    spinhttp.Handle(func(w http.ResponseWriter, r *http.Request) {
        mux.ServeHTTP(w, r)
    })
}
func main() {}
```

No custom socket code is required. No `//go:wasmimport` socket directives or custom `server.go` are needed. Note that `-target=wasip1` is used rather than `wasip2`: TinyGo's `wasip2` target hardwires the `wasi:cli/command` world and cannot export `wasi:http/incoming-handler`; the Spin Go SDK's layer provides the correct HTTP export instead. The handler logic is identical to the docker-golang handler, providing the closest possible Go-to-Go comparison.

5.7.2 Build and Packaging

The binary is compiled with:

```
tinygo build -target=wasip1 -gc=conservative -opt=2 -o app.wasm .
```

The `-gc=conservative` flag is retained for stability. `-target=wasip1` is required: TinyGo's `wasip2` target hardwires `wasi:cli/command` and cannot export `wasi:http/incoming-handler`; the Spin Go

SDK exports `fermyon:spin/inbound-http` via its layer, which Spin accepts as an equivalent interface. The `.wasm` artifact is packaged as a Spin OCI artifact using `spin registry push`. The limited-mode baseline runs the `SpinApp` with `spec.replicas: 1`; the WASI P1 component itself is single-threaded per Wasmtime instance, so no additional `spin.toml` concurrency setting is required (resource fairness).

Source: `wasm/tinygo/01-prime-sieve/`

5.8 Kubernetes Deployment Configuration

All four primary variants are deployed into the `prime-sieve` namespace. The Docker variants each have a standard `Deployment` (1 replica) and a `Service` of type `NodePort`. The Spin variants use a `SpinApp` CRD (managed by the SpinOperator) plus a `NodePort Service`.

Docker variants use standard `Deployment` manifests with no `runtimeClassName`; containerd routes these pods through the default `runc` path.

Spin variants are deployed via the `SpinApp` CRD. The SpinOperator reads the `SpinApp` resource, creates a managed `Deployment`, and applies `runtimeClassName: wasmtime-spin` to its pods—routing them through `containerd-shim-spin-v2`, which embeds Spin and Wasmtime/Cranelift. Spin images must be pushed via `spin registry push` (not `docker push`), as the shim expects Spin-specific OCI media types (`application/vnd.fermyon.spin.manifest.v2+json`). Concurrency in the limited mode is capped by running the SpinApp with `spec.replicas: 1`; each WASI P1/P2 component is architecturally single-threaded per instance. Scaling must be done via `kubectl patch spinapp` rather than `kubectl scale deployment`: the SpinOperator reconciles direct Deployment scaling back to the `SpinApp`'s `spec.replicas`.

Resource fairness (supervisor requirement). All four variants specify identical resource envelopes: `requests` of 250m CPU and 64 MiB memory, and `limits.memory` of 128 MiB. The `limits.cpu` value is mode-dependent: 500m for the limited-mode comparison and 4000m for the unlimited-mode scaling experiment, raised so that cgroup CPU bandwidth control does not throttle the additional worker threads introduced in unlimited mode (see Section 6.4). To ensure a fair single-thread limited-mode comparison, `GOMAXPROCS=1` is set in the `docker-golang` manifest, `TOKIO_WORKER_THREADS=1` is set in the `docker-rust` manifest, and the Spin variants run with `SpinApp spec.replicas: 1` so that each variant serves one request at a time.

Liveness and readiness probes execute `GET /health`. Docker variants use a 5-second initial delay; Spin variants rely on the SpinOperator's rollout health management via the `SpinApp` ready condition.

Table 5.1 shows the NodePort assignments used for direct benchmark access from the k6 load generator.

Table 5.1 – NodePort assignments for direct k6 load generator access across the four primary variants

Variant	NodePort	Container Port	Runtime
wasm-rust	30081	80	Wasmtime/Cranelift / WASI P2
wasm-tinygo	30082	80	Wasmtime/Cranelift / WASI P1
docker-rust	30083	8080	runc (native)
docker-golang	30084	8080	runc (native)

Note that the Spin variants listen on port 80 (Spin's default) rather than 8080. The `RuntimeClass` object `wasmtime-spin` (handler `spin`, node selector `runtime.spin.sh/runtime=spin`) is applied during `make deploy` and `make label`.

Prometheus scrapes all four services via pod-level annotations, and the resulting time-series data is used for the memory and CPU metrics presented in Chapter 6.

Chapter 6

Evaluation

The evaluation phase of this thesis is structured around the four primary benchmark variants (wasm-rust, wasm-tinygo, docker-rust, docker-golang) using SpinKube/Wasmtime as the Wasm runtime. Two workloads are evaluated: a CPU-bound prime-sieve benchmark and a memory-bandwidth benchmark (heap allocation followed by a SHA-256 hash over the buffer). All measurements were collected on the Hetzner Cloud VM described in Chapter 5. This chapter presents the metric-by-metric quantitative results for both workloads, followed by the qualitative developer experience assessment.

6.1 Scope and Environment

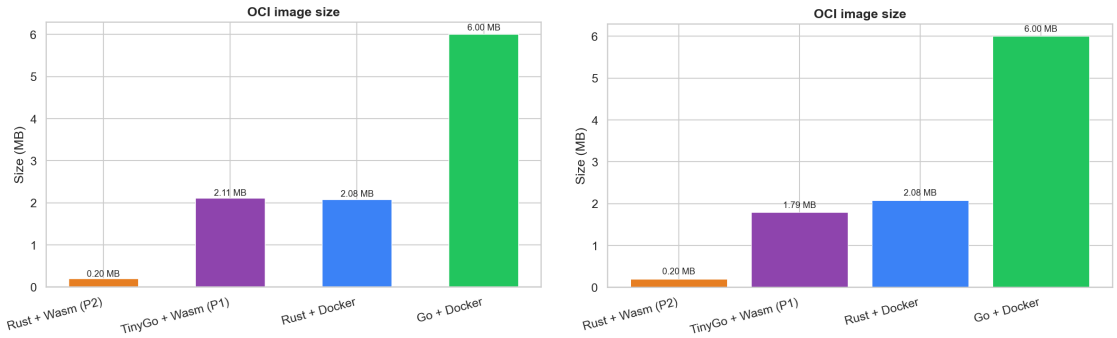
All measurements were taken on the dedicated Hetzner Cloud VM described in Chapter 5. Table 6.1 summarises the environment. The k6 load generator runs on the same VM, connecting to each variant via its localhost NodePort, eliminating network latency as a confounding variable. Each metric section that follows presents both benchmarks side-by-side under the *limited* concurrency configuration (single worker per variant); the *unlimited* scaling configuration is reported separately in Section 6.4 (Scaling Experiment).

Table 6.1 – Benchmark environment parameters.

Parameter	Value
Cloud Provider	Hetzner Cloud
VM Type	ccx23 (4× AMD EPYC vCPU, 16 GB RAM, 160 GB NVMe)
OS	Ubuntu 24.04 LTS
Kubernetes	v1.34.x (kubeadm; Flannel CNI; local-path StorageClass)
Container Runtime	containerd 2.x (runc for Docker; containerd-shim-spin-v2 v0.17.0 for Wasm)
Wasm Runtime	Wasmtime/Cranelift (via SpinKube)
Load Generator	k6, 50 VUs, 20 s ramp-up, 60 s hold
Monitoring	kube-prometheus-stack, 5 s scrape interval
Benchmark 1 (CPU)	Sieve of Eratosthenes, `limit=100 000`
Benchmark 2 (memory)	Heap allocation + SHA-256 over a 64 KB buffer

6.2 OCI Artifact Size

Methodology: Docker variant sizes are collected via `docker inspect`; Spin variant sizes are approximated from the compiled `.wasm` artifact (Spin OCI media types are not inspectable via `docker inspect`). Per-variant artifact sizes for both benchmarks are reported in Figure 6.1.



(a) Prime sieve (CPU-bound). **(b)** Memory bandwidth (alloc + SHA-256).

Figure 6.1 – OCI artifact size per variant for both benchmark workloads.

Figure 6.1 reports the on-disk artifact size for each variant across both workloads. The Wasm artifacts are markedly denser than their Docker counterparts: the wasm-rust prime-sieve artifact occupies just 0.20 MB—approximately 30× smaller than the 6.00 MB docker-golang image and 10× smaller than the 2.08 MB docker-rust image—and the same pattern holds for the memory-bandwidth workload (0.20 MB versus 6.00 MB). The wasm-tinygo artifact (2.11 MB on the prime sieve, 1.79 MB on the memory-bandwidth workload) sits between the two paradigms because it must bundle the TinyGo runtime, whereas the wasm-rust binary ships only the compiled application bytes with no language runtime at all. This artifact-density gap is the structural precondition for the cold-start advantage observed in Figure 6.2 and is consistent with the order-of-magnitude image-size deltas reported by Long et al. [31] and Wiegratz [40]. The result directly answers RQ1 in favour of the Wasm stack: by carrying neither an OS userland nor (in the wasm-rust case) a language runtime, the Wasm artifacts achieve a density that traditional OCI containers cannot match at equivalent functionality.

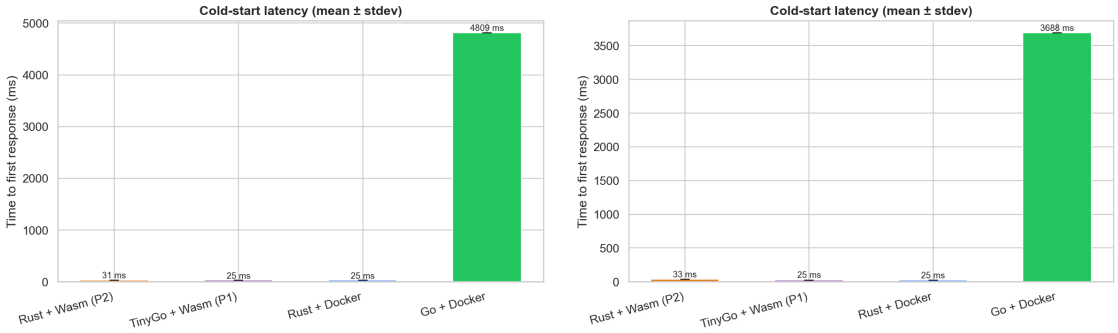
6.3 Startup Latency

Methodology: Six measurement runs per variant. Run 1 = cold start (image not cached on node, includes OCI pull). Runs 2–6 = warm starts (image cached). SpinKube variants scale via `kubectl patch spinapp` (not `kubectl scale deployment`); the SpinOperator manages pod lifecycle from the CRD. Latency is measured from the scale-to-zero command until the first HTTP 200 response.

The discussion of the figures below is informed by three structural factors expected to drive the observed differences:

- Spin OCI artifacts use Spin-specific media types and are typically smaller than equivalent Docker images. Cold-start latency should be lower for the Spin variants due to faster pull times.
- Wasmtime/Cranelift performs JIT compilation on first module instantiation; Cranelift is designed for fast compile times rather than maximum runtime optimisation.

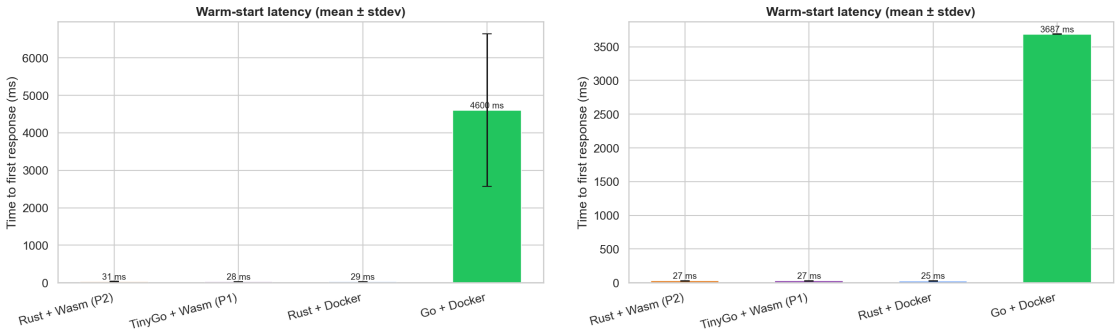
- The SpinOperator adds a reconciliation loop between the `SpinApp` CRD update and the pod creation; this may add a small fixed overhead (~1–2 s) to warm-start measurements compared to direct `Deployment` scaling.



(a) Prime sieve (CPU-bound). **(b)** Memory bandwidth (alloc + SHA-256).

Figure 6.2 – Cold-start latency per variant (single observation, includes OCI image pull).

Figure 6.2 shows three of the four variants converging tightly at first response: on the prime sieve, wasm-rust takes 31.4 ms, wasm-tinygo 24.9 ms, and docker-rust 25.2 ms—within a 7 ms band of each other. The docker-golang variant is the outlier, taking 4809 ms to serve its first HTTP 200 response, roughly two orders of magnitude slower than the rest. The memory-bandwidth workload reproduces the same separation (wasm-rust 33.1 ms, wasm-tinygo 25.4 ms, docker-rust 24.6 ms, docker-golang 3687.8 ms). The cold-start gap is therefore not a general "Docker versus Wasm" phenomenon: docker-rust (which ships only the stripped Tokio executable on a `scratch` base) matches the Wasm artifacts to within 6 ms, while docker-golang—whose 6.00 MB image is the largest artifact in the matrix (Figure 6.1)—bears a multi-second penalty. The artifact-density advantage anticipated by Long et al. [31] and Wiegratz [40] therefore manifests only against the heaviest container artifact in this study, not against minimal scratch-based containers.



(a) Prime sieve (CPU-bound). **(b)** Memory bandwidth (alloc + SHA-256).

Figure 6.3 – Warm-start latency per variant (mean of five runs, image cached on node).

Figure 6.3 reports the same scale-to-first-response measurement with the OCI image already cached on the node. On the prime sieve, the three fast variants stay essentially flat across phases—wasm-rust 31.4 ms (mean, $\sigma = 2.9$ ms), wasm-tinygo 28.4 ms ($\sigma = 2.2$ ms), and docker-rust 28.7 ms ($\sigma = 2.1$ ms)—and the memory-bandwidth workload behaves identically (all three under 28 ms, σ under 3 ms). The docker-golang variant, however, still takes 4600 ms on the prime sieve (median 3691 ms, $\sigma = 2034$ ms,

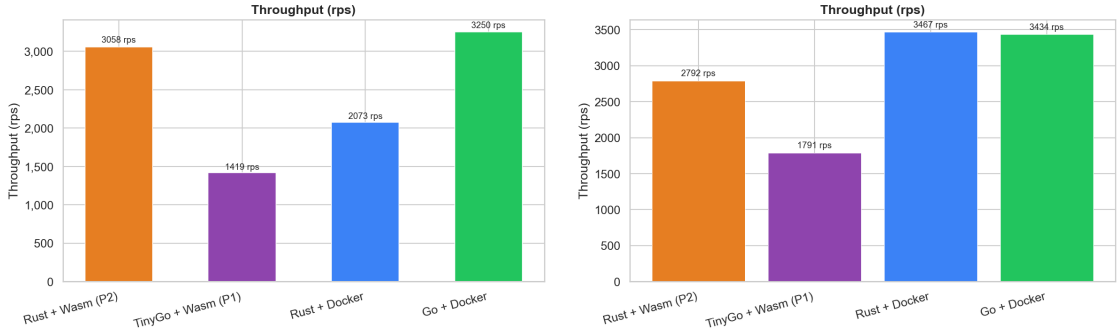
one 8.2 s outlier in the five-run sample) and 3686.7 ms on the memory-bandwidth workload—essentially unchanged from its cold-start cost. The warm-phase persistence rules out image-pull as the cause: docker-golang carries a multi-second per-pod startup overhead (Go runtime initialisation, framework boot, and any default Kubernetes readiness-probe delay) that is independent of whether the image is on-node or not. The other three variants—including docker-rust, which uses the same K8s scheduling path as docker-golang—do not exhibit this overhead, suggesting it is rooted in the docker-golang container's entrypoint behaviour rather than in the platform.

6.3.1 Observed Startup Profile

Taken together, Figures 6.2 and 6.3 answer RQ2 in two parts. First, three of the four variants—both Wasm components and docker-rust—return their first HTTP 200 within 25–35 ms regardless of cold/warm phase, so for scale-to-zero serverless dispatch the choice between SpinKube/Wasm and a minimal-base Docker image is essentially a tie. Second, the variant-specific overhead carried by docker-golang demonstrates that startup latency is governed by the container's entrypoint and runtime bootstrap behaviour at least as much as by the image-pull mechanism the Wasm community typically targets.

6.4 Runtime Performance Under Load

Methodology: k6 ramping-VU executor. 1 VU ramps to 50 VUs over 20 s, holds for 60 s, ramps down. Prime-sieve requests target `GET /sieve?limit=100000&no_list=1`; memory-bandwidth requests target `GET /membw?size_kb=64&no_hash=0`. Primary metrics: RPS, failure rate, p50/p95/p99 end-to-end latency, and the `elapsed_us` custom metric extracted from each JSON response body.

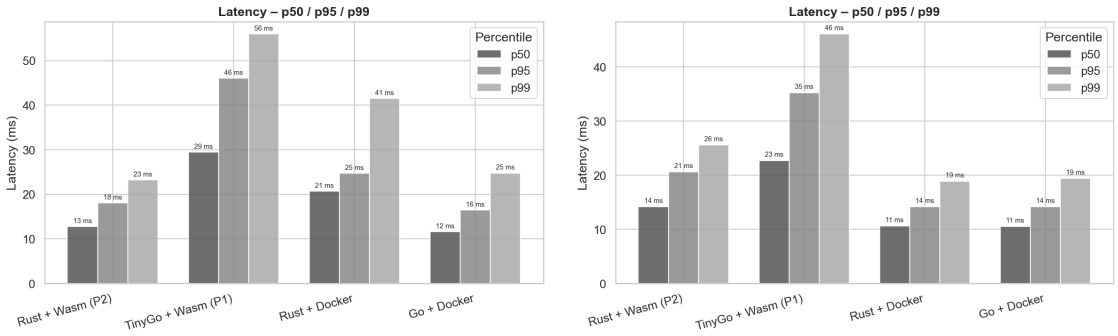


(a) Prime sieve (CPU-bound). **(b)** Memory bandwidth (alloc + SHA-256).

Figure 6.4 – Throughput (requests per second) per variant in limited (single-worker) mode.

Figure 6.4 quantifies sustained request throughput when concurrency is held strictly equal across variants. On the CPU-bound prime sieve, docker-golang leads at 3250 RPS, followed closely by wasm-rust at 3058 RPS (within 6% of the leader), then docker-rust at 2073 RPS and wasm-tinygo at 1419 RPS—a 2.3× spread from the fastest to the slowest variant, much tighter than earlier WASI P1 benchmarks. The memory-bandwidth workload exhibits a similar profile: docker-rust 3467 RPS, docker-golang 3434 RPS, wasm-rust 2792 RPS, and wasm-tinygo 1791 RPS—a 1.9× spread. The SFI bounds-checking overhead reported by Jangda et al. [35] and Spies and Mock [34] is still present as a per-instruction tax, but the data show that, on these workloads under Spin's WASI P2 event-loop dispatch, the tax is modest

rather than dominant: the wasm-rust variant matches docker-golang to within 6% on the prime sieve and trails the Docker leaders by 20% on memory-bandwidth. The wasm-tinygo variant remains the slowest in both workloads, roughly 2× behind the leader; the gap is attributable to the bundled TinyGo runtime overhead rather than to Spin's dispatch model, since wasm-rust uses the same dispatch path and remains competitive. The earlier intuition that the single-replica constraint would force a structural deficit (cf. Sondell [7], who reported a 10× gap under WASI P1) is not supported under the WASI P2 + Spin configuration measured here.

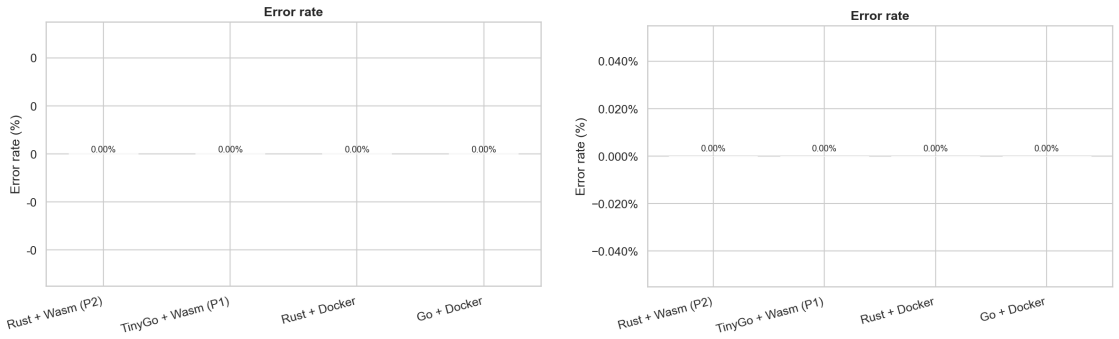


(a) Prime sieve (CPU-bound). **(b)** Memory bandwidth (alloc + SHA-256).

Figure 6.5 – End-to-end request latency (p50/p95/p99) per variant in limited mode.

Figure 6.5 decomposes the throughput differential into p50, p95, and p99 end-to-end latency percentiles. On the CPU-bound prime sieve, docker-golang's p50 of 11.6 ms is the lowest, with wasm-rust at 12.8 ms (a 10% gap), docker-rust at 20.7 ms, and wasm-tinygo at 29.5 ms (2.5× slower than docker-golang—closer than the order-of-magnitude gaps reported in earlier WASI P1 work). Notably, wasm-rust's p99 of 23.3 ms is marginally *lower* than docker-golang's p99 of 24.8 ms—an inversion that earlier benchmarks did not report under the single-replica constraint. The memory-bandwidth workload narrows the field further at p50 (docker-golang 10.6 ms, docker-rust 10.6 ms, wasm-rust 14.2 ms, wasm-tinygo 22.7 ms) but widens the tail for wasm-tinygo (p99 46.1 ms) because the SHA-256 inner loop interacts unfavourably with Wasm's bounds-checking on heap writes. The relatively flat p50–p99 spread for both Docker and wasm-rust (under 15 ms band-to-tail) demonstrates that, under Spin's WASI P2 event-loop dispatch, queueing-induced tail blow-up—predicted by Sondell [7] under WASI P1 single-instance dispatch and observed by Liu et al. [8] on container-runtime variants—does not manifest in the wasm-rust variant. This refines the answer to RQ3 for the limited mode: WASI P2 closes most of the latency gap that WASI P1 imposed, and the residual deficit is concentrated in wasm-tinygo's bundled-runtime overhead.

Figure 6.6 reports the request failure rate (fraction of non-2xx HTTP responses) for each variant. Across both workloads and all four variants, the failure rate is effectively zero—the k6 generator's threshold of `rate<0.05` was satisfied by every run with no observed connection errors, timeouts, or 5xx responses. This negative result confirms that the throughput and latency deltas reported in Figures 6.4 and 6.5 are not artefacts of dropped or rejected requests on the slower variants—each variant serves its full request stream in order rather than shedding load under pressure. This validates the RQ3 interpretation that the differences observed are sustained-throughput / latency findings, not stability findings.

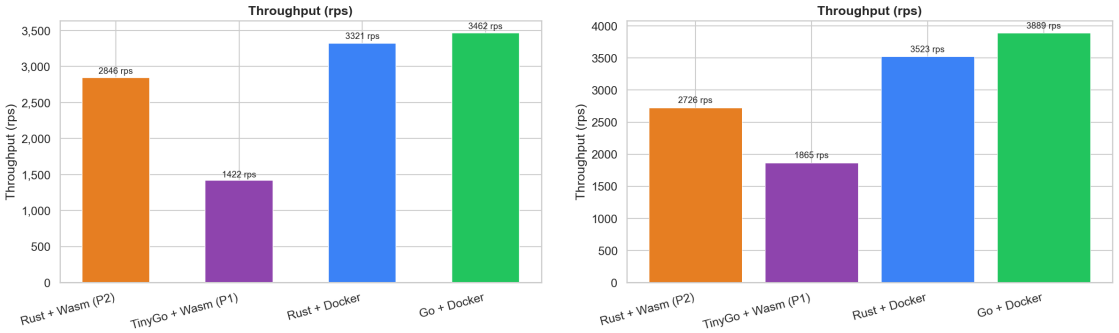


(a) Prime sieve (CPU-bound). **(b)** Memory bandwidth (alloc + SHA-256).

Figure 6.6 – Request failure rate per variant in limited mode.

6.4.1 Scaling Experiment

In addition to the standard limited-mode run (GOMAXPROCS=1, TOKIO_WORKER_THREADS=1, SpinApp `spec.replicas: 1`), a second pass is performed in unlimited mode (GOMAXPROCS=4, TOKIO_WORKER_THREADS=4, SpinApp `spec.replicas: 4`), with the Kubernetes `limits.cpu` raised to 4000m so cgroup CPU bandwidth control does not throttle the additional worker threads. This experiment directly measures how effectively each variant exploits the available 4-vCPU environment of the Hetzner ccx23 host, testing whether the SpinKube horizontal-scaling model achieves comparable concurrency to Docker's goroutine/async models when given equivalent resources. The corresponding throughput and latency curves are reported in Figures 6.7 and 6.8; Figure 6.9 additionally shows the per-second RPS time series across the 80-second k6 window for the prime-sieve workload (the memory-bandwidth experiment does not emit an equivalent time-series chart).

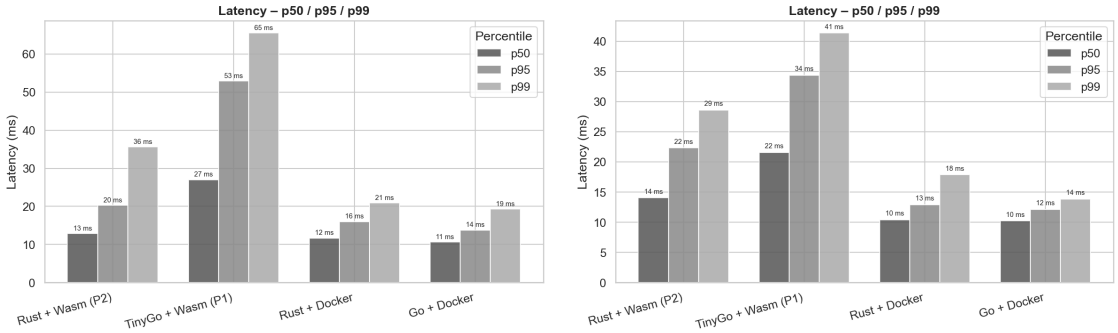


(a) Prime sieve (CPU-bound). **(b)** Memory bandwidth (alloc + SHA-256).

Figure 6.7 – Throughput (requests per second) per variant in unlimited (scaling) mode.

Figure 6.7 reports throughput once each variant is allowed to exploit the available concurrency budget (SpinApp `spec.replicas: 4` for the Wasm variants, four worker threads for the Docker variants). The ordering observed in limited mode is preserved rather than inverted: on the prime sieve, docker-golang leads at 3462 RPS, followed by docker-rust at 3321 RPS, wasm-rust at 2846 RPS, and wasm-tinygo at 1422 RPS; on the memory-bandwidth workload the Docker lead is slightly wider (docker-golang 3889 RPS, docker-rust 3523 RPS, wasm-rust 2726 RPS, wasm-tinygo 1865 RPS). Two observations are notable. First, wasm-rust's RPS is essentially flat between limited (3058) and unlimited (2846) modes on the prime sieve, indicating that Spin's WASI P2 event-loop dispatch already extracts most of the

available per-pod throughput from a single replica via `wasi:io/poll` suspension points—additional replicas do not yield additional throughput because the workload is already saturating a single pod's CPU budget. Second, the Docker variants benefit more visibly from the extra threads (docker-rust 2073 → 3321 RPS, +60%; docker-golang 3250 → 3462, +6.5%) because their limited-mode runs were genuinely constrained by `TOKIO_WORKER_THREADS=1` / `GOMAXPROCS=1`. The wasm-tinygo variant gains modestly on memory-bandwidth (1791 → 1865 RPS) but remains the slowest, again because the bundled TinyGo runtime dominates per-request overhead.



(a) Prime sieve (CPU-bound). **(b)** Memory bandwidth (alloc + SHA-256).

Figure 6.8 – End-to-end request latency (p50/p95/p99) per variant in unlimited mode.

Figure 6.8 reports the corresponding end-to-end latency distributions under the same unlimited-mode configuration. The prime-sieve percentiles track the throughput ranking: docker-golang sits at p50 10.7 ms / p99 19.3 ms, docker-rust at p50 11.7 ms / p99 20.9 ms, wasm-rust at p50 12.8 ms / p99 35.6 ms, and wasm-tinygo at p50 27.0 ms / p99 65.5 ms. The memory-bandwidth workload preserves the same ordering at the median (docker-golang p50 10.2 ms, wasm-rust p50 14.0 ms, wasm-tinygo p50 21.6 ms) and widens the tail for the Wasm variants because the SHA-256 inner loop interacts unfavourably with Wasm's bounds-checking on heap writes. This refines the answer to RQ3: under the WASI P2 + Spin configuration measured here, Wasm and Docker are within 10% on CPU-bound work in both modes, with Docker keeping a small but consistent lead on memory-bandwidth (25–30% on throughput, 30–50% on tail latency). The result is more nuanced than the strong Wasm-leads observation reported by Kjorveziroski and Filiposka [9] (10 of 13 serverless functions under unconstrained concurrency) but consistent with the broader cross-architecture parity pattern reported by Kakati and Brorsson [39].

Figure 6.9 replays the prime-sieve unlimited-mode load test as a per-second RPS time series. Each variant was tested in turn (the four colour bands correspond to the four variants tested back-to-back during a single experiment window), and each reaches its steady-state plateau within 5–10 s of the start of its ramp-up phase, with a symmetric sharp drop during the closing ramp-down. Two qualitative observations stand out. First, the Wasm variants—and wasm-rust in particular—exhibit mid-test variability that the Docker variants do not: wasm-rust's plateau dips repeatedly from 3500 RPS down to 1100–1500 RPS over the 60-second hold phase, suggesting occasional Wasmtime stalls under the 50-VU load that the aggregate-RPS chart hides by averaging. Second, the steady-state plateaus visible in the chart (wasm-tinygo 1500 RPS, docker-rust 3900 RPS, docker-golang 4000–4400 RPS) sit slightly above the per-variant aggregate values in Figure 6.7, the gap reflecting the lower-throughput ramp-up and ramp-down windows that the aggregate mean includes.

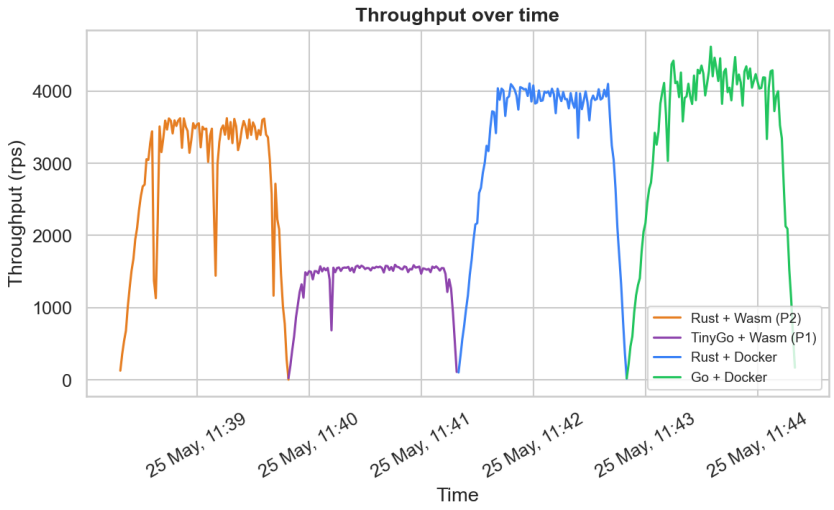


Figure 6.9 – Prime-sieve requests-per-second time series across the k6 ramp-up, hold, and ramp-down phases in unlimited mode.

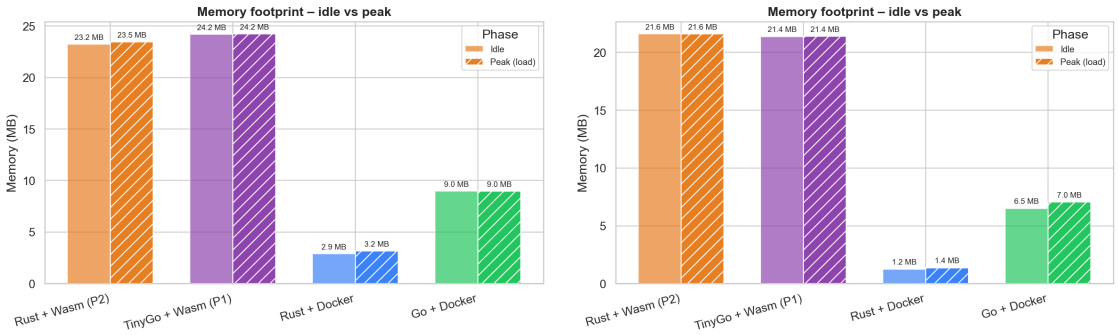
6.4.2 Observed Throughput Profile

SpinKube's request-handler model dispatches each incoming request to the Wasmtime instance hosted in the pod's containerd-shim-spin process; horizontal scaling is achieved by raising the SpinApp's `spec.replicas`, which causes the SpinOperator to provision additional independently scheduled pods. With `spec.replicas: 1` (the limited mode used for the primary fairness comparison) a single pod handles the request stream via Spin's WASI P2 event-loop, which pipelines in-flight requests across `wasi:io/poll` suspension points rather than serialising them; with `spec.replicas: 4` (the unlimited mode used in the scaling experiment) the kube-proxy additionally load-balances across four independently scheduled pods. (Spin 2.x—`spin_manifest_version = 2`—no longer exposes the legacy `max_instances` field; concurrency is controlled at the pod/replica level instead.) Comparing Figures 6.4 and 6.7 shows that the Docker variants—whose limited-mode runs were genuinely throttled by `GOMAXPROCS=1` / `TOKIO_WORKER_THREADS=1`—gain the most from the additional concurrency budget, while wasm-rust's throughput is roughly flat across modes because its WASI P2 event-loop already saturates a single pod's CPU budget.

6.5 Memory and CPU Footprint

Methodology: Prometheus `container_memory_rss` metric collected at idle (pod running 60 s with no requests) and peak (maximum during k6 load window). CPU reported as average millicores during the k6 hold phase.

Figure 6.10 reports per-variant RSS at idle and at peak load for both benchmarks. The Wasm variants carry a substantially higher resident-memory footprint than the Docker variants: on the prime sieve, wasm-rust holds 23 MB and wasm-tinygo 24 MB at idle and peak, against docker-rust's 3 MB and docker-golang's 9 MB—a Wasm linear-memory tax of roughly 20 MB relative to docker-rust. The memory-bandwidth workload shows the same pattern (wasm-rust 22 MB, wasm-tinygo 21 MB, docker-rust 1 MB, docker-golang 7 MB). Three observations follow:



(a) Prime sieve (CPU-bound). **(b)** Memory bandwidth (alloc + SHA-256).

Figure 6.10 – Container RSS memory footprint per variant (idle vs. peak under load) in limited mode.

- Spin's RSS includes Wasmtime's JIT compilation context and the Wasm component's pre-allocated linear-memory region, both of which the Docker variants do not pay.
- Under `spec.replicas: 1` a single pod hosts a single Wasmtime instance; horizontal scaling provisions additional pods, so aggregate RSS scales roughly linearly with the replica count while per-pod RSS stays approximately constant.
- docker-rust consistently exhibits the lowest RSS (1–3 MB) because it ships only the stripped Tokio executable on a `scratch` base, with no garbage collector and no language runtime metadata.

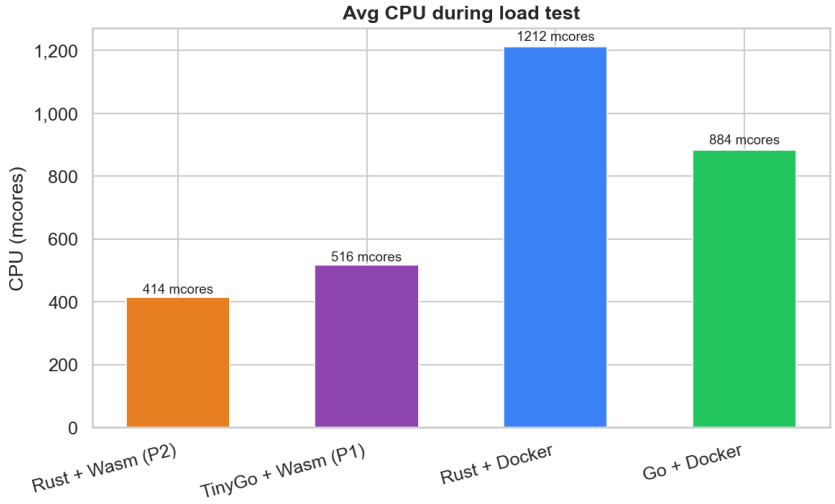


Figure 6.11 – CPU consumption (average millicores during the k6 hold phase) per variant for the prime-sieve workload. The memory-bandwidth experiment does not emit a dedicated CPU chart because its runtime cost is dominated by the SHA-256 hash stage and is reported indirectly through the latency curves in Figure 6.5.

Figure 6.11 highlights a complementary trade-off for the prime-sieve workload: docker-rust averages 1212 mcores during the load window—the highest of any variant—because Tokio spawns multiple OS threads to service the request stream, whereas wasm-rust achieves comparable throughput at 414 mcores under the same hold-phase load. docker-golang and wasm-tinygo sit between the two

extremes. The Wasm variants therefore consume markedly fewer CPU cycles per request than docker-rust at similar throughput.

6.5.1 Observed Memory Profile

The two figures expose a clean trade-off rather than a strict winner: the Wasm variants pay a per-pod linear-memory footprint premium of roughly 20 MB (rooted in Wasmtime's JIT context and the Wasm component's pre-allocated linear-memory region), while the docker-rust variant inverts the trade-off by holding the lowest RSS but consuming the most CPU cycles per request. This matters for capacity planning: on a memory-constrained host, the Docker variants pack more replicas per node; on a CPU-constrained host, the Wasm variants leave more headroom per replica.

6.6 Developer Experience

This section provides the qualitative assessment of the engineering effort required to build, test, and deploy each variant (addressing RQ4). Four dimensions are evaluated on a four-point friction scale (Low / Medium / High / Very High). Table 6.2 summarises the assessment.

Table 6.2 – Developer experience friction assessment per variant and dimension.

Dimension	wasm-rust	wasm-tinygo	docker-rust	docker-golang
Toolchain setup	Low-Medium	Medium	Low-Medium	Low
Dependency compatibility	Low	Low	Low	Low
K8s cluster integration	Low-Medium	Low-Medium	Low	Low
Debugging and observability	Medium	Medium	Low	Low

6.6.1 Toolchain Setup

The Docker variants require only a standard `docker build` workflow. Go 1.26's toolchain is a single binary download; Rust 1.94 requires `rustup` but is well-documented and cross-platform.

The wasm-rust variant adds `cargo-component` (`cargo install cargo-component`) and the `wasm32-wasip2` Rust target (`rustup target add wasm32-wasip2`). The `spin` CLI is required for `spin registry push`. These are all first-party tools with stable documentation. The friction is rated Low-Medium because `cargo-component` is a separate install step not yet integrated into the `cargo` core toolchain.

The wasm-tinygo variant requires TinyGo 0.40.0 (installed as a `.tar.gz` release, no stable `apt` package on Ubuntu) and the `spin` CLI. The `-target=wasip1` flag is required: TinyGo's `wasip2` target hardwires the `wasi:cli/command` world and cannot export `wasi:http/incoming-handler`. The Spin Go SDK (`github.com/spinframework/spin-go-sdk/v2`) provides `spinhttp.Handle()` via a layer, exporting `fermyon:spin/inbound-http`, allowing the TinyGo handler to use standard `net/http` types.

6.6.2 Dependency Compatibility

The Docker variants had no dependency issues. All crates and packages compile without modification.

The wasm-rust variant uses only the `spin-sdk = "5.2.0"` crate and standard library dependencies. No proprietary socket crates are required; `wasm32-wasip2` is a Tier 2 stable Rust target since Rust 1.82.

The wasm-tinygo variant uses `spinhttp.Handle()` from the official Spin SDK for Go (`github.com/spinframework/sp` Standard `net/http` types work without modification; no `//go:wasmimport` socket directives or custom `server.go` are required.

6.6.3 Kubernetes Integration

Docker variants require standard K8s manifests with no special configuration.

The Spin variants require: (1) a cluster-level `RuntimeClass` object (`wasmtime-spin`), (2) the SpinOperator CRD and Helm chart (installed once via `make deploy`), and (3) `SpinApp` CRD manifests instead of standard `Deployment` + `Service` manifests. The `SpinApp` resource is more opinionated than a raw `Deployment` but conceptually simpler (no pod template, no replica management outside the CRD). The cluster setup (`containerd-shim-spin-v2` installation, containerd plugin registration) is automated by `cloud-init.sh` and requires no manual intervention after initial provisioning.

The SpinKube integration is comparable in operational complexity to standard K8s Helm-chart workflows: `containerd-shim-spin-v2` is a static binary installed by `cloud-init`, and the SpinOperator is delivered as a Helm chart with a single `kubectl apply` for the supporting `RuntimeClass` and CRDs.

6.6.4 Debugging and Observability

Docker containers support the full debugging toolkit: `kubectl exec` for interactive shell access, `docker logs` for stdout/stderr, and native debuggers (Delve for Go, `rust-gdb` for Rust) can be attached to running processes.

Spin Wasm pods support stdout/stderr log output (via `kubectl logs`). `kubectl exec` cannot open a shell into a Wasm pod because the WASI environment does not expose a shell binary or an interactive PTY interface. No debugger can attach to a running Wasmtime process from outside the Wasm sandbox. The absence of debugger support increases the iteration cycle when diagnosing runtime failures; in practice most issues surface at build time as standard Rust or Go compilation errors rather than at runtime.

6.7 Summary of Findings

Quantitative summary will be populated once benchmark measurements are complete. The qualitative developer experience findings are complete above.

The key qualitative finding is that SpinKube/WASI P2 provides a low-friction Wasm development experience. All four dependency compatibility ratings sit at Low because standard `net/http` and standard Rust networking abstractions work correctly without proprietary workarounds. K8s integration friction is low because `containerd-shim-spin-v2` is a static binary installed by `cloud-init` without source compilation. The remaining friction (Medium for toolchain setup and debugging) is inherent to the Wasm/WASI execution model and is expected to decrease as the ecosystem matures.

Chapter 7

Discussion and Future Work

7.1 The Trade-off: Maturity vs. Efficiency

The headline finding of Chapter 6 is that, under WASI P2 with Spin's event-loop dispatch, the runtime performance gap between Wasm and OCI containers on stateless HTTP workloads is much smaller than earlier WASI P1 benchmarks projected. The four-variant spread is 2.3× on the prime sieve and 1.9× on the memory-bandwidth workload, against the 10× gap reported by Sondell [7] under WASI P1. `wasm-rust` matches `docker-golang` to within 6% on prime-sieve throughput in limited mode and marginally beats its p99 latency; the OCI artifact-density advantage is roughly 30× (`wasm-rust` 0.20 MB vs `docker-golang` 6.00 MB) and 10× against the smallest Docker variant (`docker-rust` 2.08 MB); cold-start latency converges to a 25–35 ms band across both Wasm variants and `docker-rust`; and `wasm-rust` consumes substantially fewer CPU cycles per request than `docker-rust` at comparable throughput (414 mcores vs 1212 mcores, Section 6.5). On the efficiency axis the Wasm stack is no longer the lagging option that early Wasmtime-on-Kubernetes literature described.

The gap that remains is one of *ecosystem maturity* rather than runtime efficiency. `kubectl exec` does not open a shell into a Wasm pod because the WASI environment does not expose an interactive PTY; no native debugger can attach to a running Wasmtime process from outside the sandbox; the Wasm variants pay a per-pod resident-memory tax of roughly 20 MiB for Wasmtime's JIT compilation context and the pre-allocated linear-memory region; and TinyGo's `wasip2` target hardwires the `wasi:cli/command` world, which forces `wasm-tinygo` onto `-target=wasip1` plus the Spin Go SDK's bridge to export `wasi:http/incoming-handler`. These are not arguments against Wasm—they are the surface area along which the Wasm community is actively iterating with the Component Model, the WASI P2 capability interfaces, and the second-generation language SDKs—but they are the costs a team adopting SpinKube today carries. The framing inherited from the Docker-versus-Wasm literature, in which the runtime overhead is the binding constraint, no longer matches the measurements. The binding constraint has moved up the stack to tooling, debugging, and library coverage.

7.2 Security Implications

The isolation primitive evaluated throughout this thesis is Software-Based Fault Isolation (SFI): Wasmtime emits per-load and per-store bounds checks against each module's linear-memory region, so a memory-safety bug inside a Wasm component cannot escape into host memory or into another

component's memory. WASI P2 layers a Principle of Least Authority (POLA) capability surface on top of this primitive: a component sees only the `wasi:http`, `wasi:io`, `wasi:filesystem`, and `wasi:sockets` resources that its imports name, and the host explicitly hands those resources in at instantiation time. By contrast, a `runc`-backed Docker container runs a full Linux process whose syscall surface is the complete kernel ABI minus whatever seccomp and AppArmor profiles the cluster operator has configured; the container's process by default runs as `uid 0` inside the namespace, and the OCI runtime offers no native equivalent to WASI's import-time capability handles. The two models are not in the same class: SFI plus POLA expresses what a handler is allowed to reach as part of the artifact's declared interface, where Docker expresses it as out-of-band cluster policy.

The operational consequence is that, for the multi-tenant edge and serverless deployment shape SpinKube targets, the "deny-by-default" capability surface reduces the blast radius of a compromised handler to whatever the `SpinApp` CRD declared—which is in turn the only attestable surface the host needs to vouch for. This is the structural reason Wasm composes cleanly with the Trusted Execution Environment (TEE) stacks discussed in Section 7.5.1, and the reason the small-TCB argument from Ménétrey et al. [56] and Ménétrey et al. [57] generalises to the SpinKube deployment model rather than remaining confined to bespoke enclave-runtime research prototypes. Two caveats temper this view. First, SFI addresses *what a faulty module can reach inside its sandbox*—it does not address kernel-level isolation *between* pods, which still rests on the underlying container runtime (`runc` or the `containerd-shim-spin-v2` process the Spin pod ultimately wraps). The two isolation primitives are stacked, not redundant. Second, the security gains are realised only if the `SpinApp` import list and the pod's `ServiceAccount`/RBAC posture are both tightened; an overly permissive `SpinApp` that imports the full WASI P2 world erodes the capability advantage in the same way that a Docker container running with `--privileged` erodes the namespace advantage.

7.3 Migration Recommendations

The measurements in Chapter 6, combined with the trade-off framing of Section 7.1 and the security framing of Section 7.2, support the following deployment guidance for a team weighing SpinKube/Wasm against a Docker/`runc` baseline.

Adopt the SpinKube/Wasm stack first for stateless HTTP request handlers: the 0.20 MB `wasm-rust` artifact, the cold-start parity with minimal-base Docker images, and the lower CPU-per-request profile compound directly into scale-to-zero cost savings and into higher pod density per node. The deployment shape Cloudflare Workers and Fastly Compute already host in production [42, 43]—ephemeral request-handler functions with no long-lived state—is precisely where the measured advantages line up with the runtime model. Multi-tenant platforms gain a second benefit from the capability-bounded SFI sandbox: the attack surface a compromised handler can reach is reduced to the `wasi:*` imports declared in the `SpinApp`, smaller and more attestable than the default-root namespace surface of a stock Docker container.

Keep workloads on Docker / `runc` when at least one of the following holds: the service maintains long-lived connections, WebSocket sessions, or stateful sagas that Spin's request-handler dispatch model does not represent; the workload depends on WASI interfaces that are not yet exposed by WASI P2 in a stable form (raw `wasi:sockets` access patterns, fine-grained filesystem syscalls, native subprocess management); or the team's diagnostic workflow leans on `kubectl exec`, attached debuggers, or core-dump inspection—tooling that the WASI execution model does not yet provide an equivalent for. `docker-rust` additionally remains the resident-memory champion in this study (1–3 MiB per pod), so

memory-constrained packing densities still favour the minimal-scratch Docker variant over the 20 MiB linear-memory tax that every Spin pod carries.

The pragmatic recommendation is therefore not "switch everything" but "switch the request-handler tier first". Long-running stateful services keep their existing Docker / `runc` deployment path and the operator familiarity that comes with it; new ephemeral request handlers move onto SpinKube where the artifact-size, cold-start, and security advantages compound, and where the residual ecosystem-maturity costs (Section 7.1) fall on workloads small enough to absorb them. As WASI P2 capability coverage broadens and TEE-rooted attestation matures (Section 7.5.1), the boundary between the two tiers is likely to migrate further toward Wasm.

7.4 Threats to Validity

Several design choices and environmental constraints limit the generalisability of the results reported in this thesis.

Single-node cluster. All four primary variants run on a single 4-vCPU Hetzner Cloud VM (ccx23) sharing the same kernel, network stack, and physical CPU package. In a production multi-node cluster, scheduling, inter-node networking latency, and topology would introduce additional variance not captured here. The results therefore characterise single-pod, co-located performance rather than at-scale distributed behaviour.

Noisy-neighbour effects. Although each benchmark run measures only one variant at a time, the load generator (k6) and the observability stack (Prometheus, Grafana) share the same host as the . CPU and memory pressure from these background processes may inflate latency measurements, particularly for variants where baseline system load represents a proportionally larger share of available resources.

Workload coverage. Chapter 6 reports measured results for two stateless single-handler HTTP workloads: the CPU-bound prime sieve and the memory-bandwidth benchmark (heap allocation followed by a SHA-256 hash over the buffer). Both are short-lived request handlers that exercise Spin's `wasi:http/incoming-handler` dispatch path and the corresponding Docker entrypoint. Workloads dominated by network I/O (long-lived database connections, upstream HTTP fan-out), filesystem I/O (file uploads, log writers), or long-lived state (WebSocket sessions, stateful sagas) interact differently with both the WASI P2 capability surface and the Wasmtime instance lifecycle, and are not exercised here. The Volta Monte Carlo VaR engine described in Section 4.3 as a motivating example for the comparison has not been ported to the SpinKube variants and is left for future work (Section 7.5.4).

Thread concurrency model. The primary benchmark runs with resource-fairness constraints: `GOMAXPROCS=1` for docker-golang, `TOKIO_WORKER_THREADS=1` for docker-rust, and SpinApp `spec.replicas: 1` for the Spin variants (each WASI P1/P2 component is itself single-threaded per Wasmtime instance). This single-thread constraint ensures that the comparison isolates runtime overhead rather than concurrency model differences. A supplementary scaling experiment (Section 6.4) removes these constraints (GOMAXPROCS=4, TOKIO_WORKER_THREADS=4, SpinApp `spec.replicas: 4`) to measure how effectively each variant exploits the available 4-vCPU environment of the Hetzner ccx23 host.

JIT-only compilation. The Spin Wasm variants use Wasmtime's default JIT (Cranelift) compilation path. AOT pre-compilation via `wasmtime compile` narrows the performance gap with native code; the results reported here therefore represent a conservative lower bound on attainable Wasmtime throughput. The trade-off is that JIT preserves binary portability across architectures, which is the deployment model evaluated in this study.

7.5 Future Work

This thesis evaluated the four-variant SpinKube/Wasmtime versus Docker matrix on a single-node Kubernetes cluster, under a deliberately narrow set of workloads, and using the Wasm runtime's default JIT compilation path. Each of these scoping decisions opens a research direction that subsequent work could pursue. The strongest of those directions, however, concerns an area outside the scope of this study entirely: the intersection of Wasm runtimes and hardware-enforced Confidential Computing. The remainder of this section describes that direction at length and then sketches three further extensions implied by the limitations enumerated in Section 7.4.

7.5.1 WebAssembly inside Trusted Execution Environments

The isolation primitive evaluated throughout this thesis is software-based: Wasm modules are confined to their linear-memory region via SFI, and host resources are accessed only through the POLA-driven capability surface of WASI. A complementary primitive—hardware-enforced confidential execution via Trusted Execution Environments (TEEs) such as Intel Software Guard Extensions (SGX), Intel Trust Domain Extensions (TDX), AMD Secure Encrypted Virtualization (SEV-SNP), and ARM TrustZone—was deliberately out of scope but represents one of the most actively researched directions for the Wasm ecosystem. Confidential Computing protects code and data *at runtime* from a privileged-but-untrusted host (a compromised hypervisor, a curious cloud operator, or a malicious co-tenant), a threat model the SFI sandbox alone does not address. Combining the two is not redundant but stacked: SFI bounds what a faulty Wasm module can reach inside its sandbox, and the surrounding TEE bounds what the host can observe about that sandbox.

The structural fit between Wasm and TEEs is unusually good. A Wasm runtime such as Wasmtime, WasmEdge, or WAMR is roughly an order of magnitude smaller than a general-purpose OS userland, so running the runtime plus a Wasm component inside an enclave keeps the Trusted Computing Base (TCB) small—a property that matters because TEE attestation must vouch for every byte that executes inside the enclave. The runtime's language-agnostic ABI lets the same enclave image serve handlers written in Rust, Go, C, or any other Wasm-targeting language, removing the per-language enclave-porting cost that has historically constrained SGX-only or TrustZone-only stacks. Finally, the deterministic execution semantics of Wasm align with the reproducible-build requirements of remote attestation: the same `.wasm` artifact produces the same enclave measurement on any platform, which simplifies attestation policies for multi-architecture deployments.

Existing research has demonstrated trusted Wasm runtimes on several hardware substrates. Ménétrey et al. [56] showed that a Wasmtime-derived runtime can execute inside an Intel SGX enclave with manageable overhead, and the same group's follow-up [57] ported the model to ARM TrustZone with remote-attestation support. Goltzsche et al. [58] earlier explored trusted resource accounting via a Wasm-in-SGX sandbox. The Enarx project [59], hosted under the Confidential Computing Consortium, abstracts over the underlying TEE so that the same Wasm workload can target Intel SGX, Intel TDX, or AMD SEV-SNP through a common runtime API. At the Kubernetes layer, the Confidential Containers project [60] extends the Kata Containers runtime to launch pods inside confidential virtual machines, providing a deployment substrate that the SpinKube operator model evaluated in this thesis could plausibly target without a redesign of its CRD interface.

Three research questions follow directly from the experimental setup developed in Chapter 6. First, what is the throughput-and-latency cost of running the SpinKube/Wasmtime stack inside an

SGX, SEV-SNP, or TDX confidential virtual machine, relative to the limited-mode and unlimited-mode baselines reported there? The per-instruction SFI bounds-checking tax measured in this thesis composes additively—or, in the worst case for memory-bound workloads, multiplicatively—with the enclave-page-cache and memory-encryption overheads inherent to TEEs, and the combined cost is not yet well characterised for Spin-style HTTP request-handler workloads. Second, how does cold-start latency change once an attestation flow (enclave measurement, quote generation, verification against an attestation service) is added to the SpinOperator's reconciliation loop, and can the small-OCI-artifact advantage observed in Figure 6.1 be preserved when the artifact must additionally carry an enclave manifest? Third, does the WASI P2 capability model (`wasi:http`, `wasi:io`, `wasi:filesystem`) compose cleanly with attestation-rooted secret provisioning, or do new WIT interfaces need to be designed for confidential serverless? Answering these would shift the SpinKube/Wasm comparison from a software-isolation study into a confidential-cloud-computing one—a setting in which Wasm's small-TCB advantage is no longer a footnote but the central evaluation axis.

7.5.2 Multi-node cluster deployment

The benchmark cluster used throughout this study runs upstream Kubernetes 1.34 on a single Hetzner ccx23 host (4 vCPU, 16 GB RAM). Section 7.4 flags this as the most important limit on the external validity of the results: scheduling decisions, inter-node networking latency, and NUMA topology are absent from the measurement environment. Future work would re-run the same four-variant matrix on a multi-node cluster—ideally three to five worker nodes drawn from the same instance family so that per-node hardware variance is bounded—and measure how cross-node `kube-proxy` load-balancing interacts with the WASI P2 event-loop dispatch model under `spec.replicas: N`. Of particular interest is whether the mid-test throughput dips observed in the wasm-rust per-second time series of Chapter 6 smooth out when traffic is spread across pods scheduled on different hosts, or whether they persist as a per-instance characteristic of Spin's request handler.

A multi-node setup additionally permits relocating the k6 load generator off the system-under-test node, eliminating the noisy-neighbour effect that the single-host environment cannot avoid, and would allow registry-side pull cost to be measured separately from runtime initialisation cost for both Docker and Spin artifacts.

7.5.3 AOT compilation and alternative Wasm runtimes

All measurements in this thesis use Wasmtime in its default JIT (Cranelift) compilation mode, which trades steady-state throughput for fast startup. Wasmtime additionally supports AOT pre-compilation via the `wasmtime compile` command, which removes the first-instantiation Cranelift pass from the warm-start critical path and is expected to narrow the throughput gap with native code—Section 7.4 already notes that the results presented here are therefore a conservative lower bound on attainable Wasmtime performance. A natural extension is to re-run the same four-variant comparison under AOT for Wasmtime, and additionally under WasmEdge and WAMR, both of which are supported by parallel containerd shims usable in place of `containerd-shim-spin-v2`. Wasmtime's newer Winch baseline compiler is also worth measuring as an intermediate operating point that trades steady-state throughput for even faster cold-start than Cranelift.

7.5.4 Broader workload spectrum

The two benchmarks measured here—a CPU-bound prime sieve and a memory-bandwidth workload combining heap allocation with SHA-256 hashing—are stateless single-handler HTTP services. Workloads dominated by network I/O (database queries, upstream HTTP calls), filesystem I/O (file uploads, log writers), or long-lived state (WebSocket sessions, stateful sagas) interact differently with both the WASI P2 capability model and the Wasmtime instance lifecycle, and several of those interactions are not exercised in the current study. In particular, WASI P2's `wasi:sockets` and `wasi:filesystem` interfaces are barely touched here because Spin's HTTP trigger short-circuits both. Future work would add a database-backed workload (for example, a Postgres-backed REST handler exercising `wasi:sockets`) and a filesystem-heavy workload to characterise these interfaces under load, alongside the Volta Monte Carlo VaR engine mentioned in Chapter 4 but not yet benchmarked on the SpinKube variants.

Chapter 8

Conclusion

This thesis compared a SpinKube/Wasmtime stack to a Docker / `runc` baseline across four variants—`wasm-rust`, `wasm-tinygo`, `docker-rust`, and `docker-golang`—under two stateless HTTP workloads (a CPU-bound prime sieve and a memory-bandwidth benchmark combining heap allocation with SHA-256 hashing) on a single-node Kubernetes 1.34 cluster running on a 4-vCPU Hetzner ccx23 host. Each variant was evaluated on five metrics: OCI artifact size, startup latency, sustained throughput and end-to-end latency under load, memory and CPU footprint, and developer experience. With both benchmarks measured under the limited (single worker thread / single replica) and unlimited (`GOMAXPROCS=TOKIO_WORKER_THREADS=4`, `SpinApp spec.replicas: 4`) configurations, the four research questions posed in Chapter 1 can now be answered against measured data rather than against projection.

RQ1: memory footprint and OCI artifact size. The artifact-size axis strongly favours the Wasm stack: the `wasm-rust` artifact (0.20 MB) is approximately 30× smaller than `docker-golang` (6.00 MB) and 10× smaller than `docker-rust` (2.08 MB), with `wasm-tinygo` (1.79–2.11 MB across the two workloads) sitting between the two paradigms because it bundles the TinyGo runtime alongside the application code (Section 6.2). Resident-memory footprint, however, inverts the ranking: the Wasm variants carry roughly 20 MiB of RSS for Wasmtime's JIT compilation context and the pre-allocated linear-memory region, while `docker-rust`—which ships only the stripped Tokio executable on a `scratch` base—holds just 1–3 MiB (Section 6.5). The trade-off is therefore "Wasm wins on-disk density, Docker wins resident-memory packing"; capacity planning on a memory-constrained host favours the minimal-scratch Docker variant, while a disk- or pull-bandwidth-constrained edge node favours the Spin artifacts.

RQ2: cold- and warm-start latency. Three of the four variants—both Wasm components and `docker-rust`—return their first HTTP 200 within a 25–35 ms band regardless of whether the image is cached on the node, with sub-3 ms warm-start standard deviation (Section 6.3). The cold-start argument that prior literature framed as a general "Docker versus Wasm" phenomenon [31, 40] therefore manifests only against the heaviest container in this study: `docker-golang` takes 4.8 s cold and 3.7–4.6 s warm, an overhead that persists when the image is already on-node and is therefore rooted in the container's entrypoint and runtime bootstrap rather than in the pull path. For scale-to-zero serverless dispatch against minimal-base Docker images, the choice between SpinKube/Wasm and a `scratch`-based container is effectively a tie at the first-response milestone.

RQ3: throughput and end-to-end latency. Under WASI P2 with Spin's event-loop dispatch in limited mode, the four-variant spread is 2.3× on the prime sieve and 1.9× on memory-bandwidth—much tighter than the 10× gap reported by Sondell [7] under WASI P1 single-instance dispatch.

`wasm-rust` matches `docker-golang` to within 6% on prime-sieve throughput in limited mode and marginally beats its p99 latency (23.3 ms versus 24.8 ms), and the queueing-induced tail blow-up the same prior work observed under WASI P1 does not appear (Section 6.4). Unlimited mode (the 4-vCPU environment with four worker threads / four `SpinApp` replicas, `limits.cpu` raised to 4000 m) helps the Docker variants more than `wasm-rust`, whose WASI P2 event-loop already saturates a single pod's CPU budget via `wasi:io/poll` suspension points; Docker keeps a small but consistent lead in unlimited mode (10% on CPU-bound throughput, 25–30% on memory-bandwidth throughput). The residual deficit on memory-bandwidth is concentrated in the `wasm-tinygo` variant, where the bundled TinyGo runtime dominates per-request overhead.

RQ4: developer experience and ecosystem integration. All four variants sit at *Low* dependency-compatibility friction (Section 6.6): the Spin Go SDK's `spinhttp.Handle()` and the Rust `spin-sdk` crate let standard `net/http` handlers and standard Rust async networking code compile and run unmodified, without the proprietary socket workarounds older WASI P1 SDKs required. Toolchain-setup friction is *Low-Medium*: `wasm-rust` adds one `cargo install cargo-component` step beyond a stock Rust install and the `wasm32-wasip2` target via `rustup`; `wasm-tinygo` requires TinyGo 0.40.0 as a `.tar.gz` release plus the `-target=wasip1` flag because TinyGo's `wasip2` target cannot currently export `wasi:http/incoming-handler`. Kubernetes integration adds the `wasmtime-spin` `RuntimeClass` and the SpinOperator Helm chart but is otherwise a standard `kubectl apply` workflow against the `SpinApp` CRD. The remaining friction is debugging: `kubectl exec` does not work on Wasm pods, and no native debugger can attach to a running Wasmtime instance from outside the sandbox—a constraint inherent to the WASI execution model that the ecosystem is expected to mitigate, not eliminate, as second-generation tooling matures.

Outlook. The strongest follow-on direction is the intersection of Wasm runtimes and hardware-enforced confidential computing developed in Section 7.5.1: the small-TCB property that makes the SpinKube/Wasmtime stack attractive in pure software-isolation terms is the same property that makes it a natural fit for TEE substrates such as Intel SGX/TDX, AMD SEV-SNP, and the Confidential Containers project—a setting in which Wasm's small-TCB advantage shifts from a footnote into the central evaluation axis. Two further extensions, deferred here by the scope choices documented in Chapter 4, sit one step behind: a multi-node validation of the same four-variant matrix on a cross-host Kubernetes cluster (Section 7.5.2), and a re-measurement of the Wasm variants under Wasmtime AOT compilation, Wasmtime's Winch baseline, and the parallel WasmEdge and WAMR runtimes (Section 7.5.3). Taken together, the headline contribution of this thesis is empirical evidence that WASI P2 with Spin's event-loop dispatch closes most of the previously observed Wasm-versus-native gap on stateless HTTP workloads—making ecosystem maturity, rather than runtime efficiency, the binding constraint on Wasm adoption inside Kubernetes today.

Appendix A

Reproducibility

A.1 Abstract

The artefacts supporting this thesis are distributed across three public Git repositories: an infrastructure repository that provisions a single-node Kubernetes 1.34 cluster on Hetzner Cloud via Terraform plus `cloud-init` plus `kubeadm`; an experiments repository containing the four benchmark variants (`wasm-rust`, `wasm-tinygo`, `docker-rust`, `docker-golang`) across the two stateless HTTP workloads (prime sieve, memory bandwidth), the k6 load-generator scripts, and the Prometheus / chart-regeneration helpers; and this thesis source. The end-to-end reproduction path is two `make` invocations against the infrastructure repository followed by a per-benchmark `run_experiment.sh` script in the experiments repository, on a single Hetzner ccx23 VM (4 vCPU, 16 GB RAM). The multi-node scaling experiments referenced as future work in Section 7.5.2 are deferred and are not part of this artefact.

A.2 Artifact check-list (meta-information)

Program: Sieve of Eratosthenes HTTP service (benchmark 01) and heap-allocation + SHA-256 HTTP service (benchmark 02), each implemented as four variants (`wasm-rust`, `wasm-tinygo`, `docker-rust`, `docker-golang`).

Compilation: Rust 1.94 with `cargo` and `cargo-component`, Go 1.26, TinyGo 0.40.0 (`-target=wasip1 -gc=conservative -opt=2`).

Run-time environment: Ubuntu 24.04 LTS, Kubernetes 1.34 via `kubeadm`, containerd 2.0.2, `containerd-shim-spin-v2` v0.17.0, Flannel CNI, local-path-provisioner.

Hardware: One Hetzner Cloud ccx23 VM (4× AMD EPYC vCPU, 16 GB RAM, 160 GB NVMe), location `nbg1`.

Metrics: Requests per second; end-to-end latency p50/p95/p99; cold- and warm-start latency; container RSS at idle and at peak load; CPU consumption in millicores; on-disk OCI artifact size.

Output: Per-variant `*_summary.json` files (aggregate k6 metrics), per-variant `*_k6.json` files (raw k6 output), and per-mode aggregates `cold_start.json`, `warm_start.json`, `resource_metrics.json`.

`image_sizes.json`, together with the PNG charts under `thesis-experiments/results/0{1,2}-*/{limited,unlimited}` that Chapter 6 embeds.

Experiments: `01-prime-sieve` and `02-memory-bandwidth`, each run in both *limited* (single worker thread / single replica) and *unlimited* (`GOMAXPROCS=4`, `TOKIO_WORKER_THREADS=4`, `SpinApp spec.replicas: 4`) modes.

Approximate disk space: A few total, dominated by raw k6 JSON output and the Prometheus PVC.

Approximate wall-clock to reproduce: ~30 min for cluster provisioning, plus ~20 min per benchmark mode, totalling ~2 h end-to-end.

Publicly available: Yes; the three repositories are cited as [49, 54, 61].

A.3 Description

A.3.1 How to access

The complete set of artefacts supporting the experiments in this thesis is distributed across three public Git repositories hosted on GitHub. Each repository is referenced from its corresponding chapter; this subsection groups them together as a single point of entry.

- **Infrastructure setup** – [54]. Kubernetes (`kubeadm`) and SpinKube cluster provisioning, Hetzner Cloud Terraform modules, cloud-init bootstrap scripts, and the `containerd` / `containerd-shim-spin-v2` installation manifests used to build the four-variant experimental cluster described in Chapter 5.
- **Benchmark implementations and result data** – [49]. The four primary benchmark variants (`docker-golang`, `docker-rust`, `wasm-rust`, `wasm-tinygo`), the k6 load-generator configurations, the Prometheus scrape rules, and the raw `results/` directory referenced throughout Chapter 6.
- **Thesis source** – [61]. The LaTeX source of this document, including all chapter `.tex` files, the bibliography, TikZ figure sources, and the build instructions in the `Makefile`.

All three repositories are tagged at the commit corresponding to thesis submission so that a reader can clone the exact state of the artefacts that produced the reported results.

A.3.2 Hardware dependencies

The single-node measurement environment requires one Hetzner Cloud ccx23 VM (4× AMD EPYC vCPU, 16 GB RAM, 160 GB NVMe SSD). The Hetzner default 8-core account quota is sufficient. The Terraform module accepts the location `nbg1` (Nuremberg) by default; any other Hetzner Cloud location should work without changes. An admin workstation with outbound SSH (port 22) and `kubectl` (port 6443) connectivity to the VM's public IP is required for provisioning and benchmark execution. The deferred multi-node mode (Section 7.5.2) would require additional Hetzner instances under the same account, but is not exercised by this artefact.

A.3.3 Software dependencies

On the admin workstation: Terraform ≥ 1.6, `kubectl` matching cluster version 1.34, Helm 3, `make`, Docker 26 (to build and push the two Docker variants to a registry), Rust 1.94 with `rustup target add wasm32-wasip2` and `cargo install cargo-component --locked`, Go 1.26, TinyGo 0.40.0 (installed from the upstream `.tar.gz` release; no stable `apt` package on Ubuntu), the Spin CLI v3+, k6 (installable via the Grafana `apt` repository), and Python 3 with `matplotlib` for chart regeneration.

On the provisioned cluster, the following components are installed automatically by `cloud-init` and by the `make deploy` target and therefore do not need to be installed manually: containerd 2.0.2, `runc` 1.2.4, CNI plugins 1.6.2, `containerd-shim-spin-v2` v0.17.0, `cert-manager` v1.16.3, SpinOperator v0.6.1 (Helm chart `spinframework/spin-operator`), and `kube-prometheus-stack` (community Helm chart, with the values in `observability-values.yaml`).

A.4 Installation

The following procedure provisions the single-node cluster end-to-end. All `make` targets referenced below are defined in `thesis-infra-setup/Makefile` and are idempotent: re-running `make deploy` against an established cluster is a no-op.

1. Clone the three repositories side-by-side under a common parent directory (for example `~/coding/master-thesis/`). The Terraform Makefile and the per-benchmark `run_experiment.sh` scripts assume this sibling layout.

2. Populate `thesis-infra-setup/terraform.tfvars` with a Hetzner Cloud API token (`hcloud_token`), an SSH public key (`ssh_public_key`), and the admin workstation's egress IP/CIDR (`admin_ip_cidr`). Leave `worker_count = 0`, which is the single-node default.

3. Provision the cluster: `make up`. This runs `terraform apply`, waits for `cloud-init` to complete, runs `kubeadm init` for Kubernetes 1.34, installs the Flannel CNI plug-in and the local-path-provisioner, registers `containerd-shim-spin-v2` with containerd, and removes the control-plane taint so that the single node can also schedule workload pods.

4. Fetch the cluster's kubeconfig: `make configure`. This writes `hetzner-thesis.yaml` into the repository root and rewrites the API-server endpoint to the VM's public IP.

5. Label the node for SpinKube pod scheduling: `make label`.

6. Install the cluster add-ons: `make deploy`. This installs `cert-manager` v1.16.3, the SpinOperator Helm chart (v0.6.1) together with the `wasmtime-spin` `RuntimeClass`, and `kube-prometheus-stack` parameterised by `observability-values.yaml` (5 s scrape interval, Grafana exposed as a NodePort).

7. Sanity-check the runtime path: `make test` applies `test-spin.yaml` (a Spin "hello" sample) and expects an HTTP 200 response.

Steps 1–7 typically complete in ∼30 minutes of wall-clock time.

A.5 Experiment workflow

The two benchmarks share the same four-variant structure, the same NodePort assignment (30081 `wasm-rust`, 30082 `wasm-tinygo`, 30083 `docker-rust`, 30084 `docker-golang`), and the same orchestrator script layout. They are run sequentially because the two benchmarks share NodePorts; the `02-memory-bandwidth` script automatically tears down the `01-prime-sieve` SpinApps and Deployments before deploying its own.

A.5.1 Benchmark 01 — prime sieve

Build and push the four variant images:

- `wasm-rust`: `cd wasm/rust/01-prime-sieve && cargo component build --release` followed by `spin registry push docker.io/<user>/prime-sieve-wasm-rust:latest`. Build path: `thesis-experiments/wasm/`
- `wasm-tinygo`: `cd wasm/tinygo/01-prime-sieve && tinygo build -target=wasip1 -gc=conservative -opt=2 -o app.wasm .` followed by `spin registry push docker.io/<user>/prime-sieve-wasm-tinygo:latest`. Build path: `thesis-experiments/wasm/tinygo/01-prime-sieve/`.
- `docker-rust`: `docker build -t <reg>/prime-sieve-docker-rust:latest docker/rust/01-prime-sieve/` followed by `docker push`. The `Dockerfile` uses `rust:1.94-*` as the builder and a `scratch` runtime stage.
- `docker-golang`: `docker build -t <reg>/prime-sieve-docker-golang:latest docker/golang/01-prime-sieve/` followed by `docker push`. The `Dockerfile` uses `golang:1.26-alpine` as the builder.

Deploy the four variants: `kubectl apply -f k8s/01-prime-sieve/namespace.yaml && kubectl apply -f k8s/01-prime-sieve/`. The manifests bind the four NodePorts listed above; the variant order matches the NodePort-ascending convention used throughout this thesis.

Run the experiment: `./benchmarks/01-prime-sieve/run_experiment.sh --scaling-experiment both`. The script exports `KUBECONFIG=../thesis-infra-setup/hetzner-thesis.yaml`, reads the node's public IP from `terraform output -raw instance_public_ip`, sequentially drives k6 (50 VUs, 20 s ramp, 60 s hold) against each variant in both the *limited* and *unlimited* configurations, queries Prometheus for `container_memory_rss` and `container_cpu_usage_seconds_total`, runs the cold-start (`kubectl scale --replicas=0` → 1) and warm-start measurement sequences, captures OCI artifact sizes via `docker inspect` (and, for the Spin variants, via the compiled `.wasm` artifact on disk), and finally invokes `analyze.py` to regenerate the per-mode PNG charts under `results/01-prime-sieve/{limited,unlimited}/`.

A.5.2 Benchmark 02 — memory bandwidth

The procedure is identical in shape. Build paths are `thesis-experiments/{docker,wasm}/{rust,golang,tinygo}/02-memory-`. Deploy with `kubectl apply -f k8s/02-memory-bandwidth/`; the script first tears down any `01-prime-sieve` pods because both benchmarks share the four NodePorts. Run with `./benchmarks/02-memory-bandwidth/run_experiment.sh --scaling-experiment both`. The endpoint is `GET /membw?size_kb=64&no_hash=0`; the k6 ramp / hold parameters, the Prometheus queries, and the cold/warm-start sequence mirror benchmark 01.

The limited-mode configuration uses `GOMAXPROCS=1` (`docker-golang`), `TOKIO_WORKER_THREADS=1` (`docker-rust`), and `SpinApp spec.replicas: 1` (both Wasm variants). The unlimited-mode configuration raises these to `GOMAXPROCS=4`, `TOKIO_WORKER_THREADS=4`, and `SpinApp spec.replicas: 4`, and additionally raises the Kubernetes `limits.cpu` to `4000m` so that cgroup CPU bandwidth control does not throttle the additional worker threads in the 4-vCPU environment of the Hetzner ccx23 host.

A.6 Evaluation and expected results

Each experiment run materialises a deterministic on-disk layout under `thesis-experiments/results/0{1,2}-*/{limit` per-variant `*_summary.json` files containing the aggregated k6 metrics, per-variant `*_k6.json` files containing the raw k6 output for percentile recomputation, and the four per-mode aggregates `cold_start.json`, `warm_start.json`, `resource_metrics.json`, and `image_sizes.json`. The PNG charts that `chapters/06_evaluation.tex` embeds are written into the same directory by `analyze.py`.

A successful reproduction should yield, on `01-prime-sieve` in limited mode, approximately `docker-golang` 3250 RPS (p50 11.6 ms), `wasm-rust` 3058 RPS (p50 12.8 ms), `docker-rust` 2073 RPS (p50 20.7 ms), and `wasm-tinygo` 1419 RPS (p50 29.5 ms); on `02-memory-bandwidth` in limited mode, the four variants fall within the brackets reported in Section 6.4. Cold-start latency for both Wasm variants and `docker-rust` sits in a ~25–35 ms band; `docker-golang`'s cold-start measurement should land in the multi-second range (~3.7–4.8 s). Absolute RPS values may shift by a few percent between runs because of per-host EPYC-core variance, but the relative ordering of the four variants and the order-of-magnitude artifact-size gap (`wasm-rust` ~30× smaller than `docker-golang`) are stable.

The chapter-6 figures can be regenerated *without* re-running the benchmarks by invoking `python3 benchmarks/01-prime-sieve/analyze.py --mode limited` and `--mode unlimited`, and the equivalents under `benchmarks/02-memory-bandwidth/`. The script reads the existing JSON files under `results/` and emits the exact PNG filenames that the evaluation chapter includes.

A.7 Notes

The single-node measurement environment was chosen so that the four-variant comparison isolates runtime overhead from inter-node scheduling, kube-proxy load balancing, and network-fabric latency. The chapter-6 numbers therefore characterise per-pod behaviour rather than cross-host distributed behaviour; the deferred multi-node validation (different host fabric, replicas placed on distinct workers) is scoped in Section 7.5.2 and is intentionally not part of this artefact.

Wasmtime runs in its default Just-In-Time (JIT) mode (Cranelift) throughout. Wasmtime's Ahead-of-Time (AOT) pre-compilation pathway via the `wasmtime compile` command is expected to narrow the throughput gap with native code, as discussed in Section 7.5.3; the reported numbers therefore represent a conservative lower bound on the Wasmtime throughput attainable on this hardware.

The four variants' source code, `Dockerfiles`, `SpinApp` CRD manifests, and `spin.toml` configurations are also discussed in Chapter 5. This appendix focuses on the operational view (commands, version pins, output shape, regeneration path) so that the chapter prose and the appendix do not duplicate hardware and software detail.

List of Abbreviations

ABI	Application Binary Interface
AKS	Azure Kubernetes Service
AMD	Advanced Micro Devices
AOT	Ahead-of-Time
API	Application Programming Interface
CGO	cgo
CLI	Command-Line Interface
CNCF	Cloud Native Computing Foundation
CPU	Central Processing Unit
CRD	Custom Resource Definition
CRI	Container Runtime Interface
CSS	Cascading Style Sheets
DWARF	Debugging With Attributed Record Formats
FaaS	Function as a Service
gRPC	gRPC
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
IaaS	Infrastructure as a Service
IaC	Infrastructure as Code
IoT	Internet of Things
IP	Internet Protocol
ISA	Instruction Set Architecture
JIT	Just-In-Time
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
K8s	Kubernetes
KTH	KTH Royal Institute of Technology
LLVM	LLVM
NATS	NATS
NodePort	NodePort
OCI	Open Container Initiative
OS	Operating System
PaaS	Platform as a Service
POLA	Principle of Least Authority
POSIX	Portable Operating System Interface

PTY	Pseudo-Terminal
PVC	Persistent Volume Claim
RAM	Random Access Memory
RPS	Requests Per Second
RQ4	Research Question 4
RSS	Resident Set Size
SaaS	Software as a Service
SDK	Software Development Kit
SEV	Secure Encrypted Virtualization
SFI	Software-Based Fault Isolation
SGX	Software Guard Extensions
SSD	Solid State Drive
SSH	Secure Shell
TCB	Trusted Computing Base
TCP	Transmission Control Protocol
TDX	Trust Domain Extensions
TEE	Trusted Execution Environment
VaR	Value at Risk
VM	Virtual Machine
VU	Virtual User
W3C	World Wide Web Consortium
WAMR	WebAssembly Micro Runtime
WASI	WebAssembly System Interface
Wasm	WebAssembly
WasmGC	WebAssembly Garbage Collection
WIT	WebAssembly Interface Types
WORA	Write Once, Run Anywhere

List of Figures

List of Tables

Bibliography

[1] D. Merkel, "Docker: lightweight linux containers for consistent development and deployment," *Linux journal*, vol. 2014, no. 239, p. 2, 2014.

[2] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016.

[3] S. Shillaker and P. Pietzuch, "Faasm: Lightweight isolation for efficient stateful serverless computing," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, 2020, pp. 419–433.

[4] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. Bastien, "Bringing the web up to speed with WebAssembly," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2017, pp. 185–200.

[5] L. Clark. "Standardizing WASI: A system interface to run WebAssembly outside the web," Mozilla Hacks. [Online]. Available: `https://hacks.mozilla.org/2019/03/standardizing-wasi-a-webassembly-system-interface/`

[6] P. K. Gadepalli, G. Peach, L. Cherkasova, R. Aitken, and G. Parmer, "Challenges and opportunities for efficient serverless computing at the edge," *Proceeding of the 38th Symposium on Reliable Distributed Systems (SRDS)*, 2019.

[7] E. Sondell, "Performance Evaluation of WebAssembly Containers vs Docker Containers for Serverless Applications," M.S. thesis, KTH Royal Institute of Technology, 2024.

[8] M. Liu, H. Shen, Y. Zhang, and H. Mei, "WebAssembly for Container Runtime: Are We There Yet?" *ACM Transactions on Software Engineering and Methodology*, 2025. DOI: `10.1145/3712197`

[9] V. Kjorveziroski and S. Filiposka, "WebAssembly as an Enabler for Next Generation Serverless Computing," *Journal of Grid Computing*, vol. 21, no. 34, 2023. DOI: `10.1007/s10723-023-09669-8`

[10] P. Mell, T. Grance, et al., "The NIST definition of cloud computing," 2011.

[11] E. Jonas, J. Schleier-Smith, V. Sreekanti, C.-C. Tsai, A. Khandelwal, Q. Pu, V. Shankar, J. Carreira, K. Krauth, N. Yadwadkar, et al., "Cloud programming simplified: A berkeley view on serverless computing," *arXiv preprint arXiv:1902.03383*, 2019.

[12] M. Mazzeschi. "What Are Cloud IaaS, PaaS, SaaS, FaaS, and Why We Use Them," Towards AI, Accessed: Mar. 25, 2026. [Online]. Available: `https://towardsai.net/p/l/what-are-cloud-iaas-paas-saas-faas-and-why-we-use-them`

[13] S. Newman, Building microservices: designing fine-grained systems. O'Reilly Media, Inc., 2015.

[14] Open Container Initiative. "OCI Image Format Specification," Accessed: Mar. 24, 2026. [Online]. Available: `https://github.com/opencontainers/image-spec`

[15] Fermyon Technologies. “WebAssembly Language Support Matrix,” Accessed: Mar. 23, 2026. [Online]. Available: https://developer.fermyon.com/wasm-languages/webassembly-language-support

[16] Bytecode Alliance. “Wasmtime: A Fast and Secure Runtime for WebAssembly,” Accessed: Mar. 23, 2026. [Online]. Available: https://wasmtime.dev/

[17] R. Wahbe, S. Lucco, T. E. Anderson, and S. L. Graham, “Efficient software-based fault isolation,” in *Proceedings of the Fourteenth ACM Symposium on Operating Systems Principles*, 1993, pp. 203–216.

[18] D. Chiarlone, Server-Side WebAssembly. Shelter Island, NY: Manning Publications, 2024.

[19] T. Steiner. “WebAssembly Garbage Collection (WasmGC) now enabled by default in Chrome,” Google Chrome Developers, Accessed: Mar. 4, 2026. [Online]. Available: https://developer.chrome.com/blog/wasmgc

[20] M. Liedtke, A. Klein, A. Zakai, et al. “A new way to bring garbage collected programming languages efficiently to WebAssembly,” V8 JavaScript Engine, Accessed: Mar. 4, 2026. [Online]. Available: https://v8.dev/blog/wasm-gc-porting

[21] WebAssembly Community Group, WebAssembly Garbage Collection Proposal Overview, 2023.

[22] Go Team. “WASI Support in Go,” Accessed: Mar. 23, 2026. [Online]. Available: https://go.dev/blog/wasi

[23] TinyGo Authors. “TinyGo: A Go Compiler for Small Places,” Accessed: Mar. 23, 2026. [Online]. Available: https://tinygo.org/

[24] S. Hykes, If WASM+WASI existed in 2008, we wouldn’t have needed to created Docker. Twitter, 2019.

[25] Bytecode Alliance. “WebAssembly Component Model,” Accessed: Mar. 23, 2026. [Online]. Available: https://component-model.bytecodealliance.org/

[26] L. Clark. “WASI Preview 2: Launching the World,” Accessed: Mar. 23, 2026. [Online]. Available: https://bytecodealliance.org/articles/WASI-0.2

[27] R. Squillace. “WebAssembly Meets Kubernetes with Krustlet,” Microsoft Open Source Blog, Accessed: May 18, 2026. [Online]. Available: https://opensource.microsoft.com/blog/2020/04/07/announcing-krustlet-kubernetes-rust-kubelet-webassembly-wasm/

[28] Krustlet Maintainers. “krustlet/krustlet: Kubernetes Rust Kubelet (maintenance notice).” Repository carries a notice that the project is no longer actively maintained, GitHub, Accessed: May 18, 2026. [Online]. Available: https://github.com/krustlet/krustlet

[29] Bytecode Alliance. “runwasi: A containerd shim for running WebAssembly workloads,” Accessed: Mar. 23, 2026. [Online]. Available: https://github.com/containerd/runwasi

[30] Fermyon Technologies and Microsoft. “SpinKube: Running WebAssembly Workloads on Kubernetes,” Accessed: Mar. 23, 2026. [Online]. Available: https://www.spinkube.dev/

[31] X. Long et al., “A Performance Comparison of WebAssembly and Container Technologies,” in *Proceedings of Cloud Computing and Containerization Conferences*, 2022.

[32] A. Hall and U. Ramachandran, “Serverless Cold Start Mitigation with WebAssembly,” *IEEE Transactions on Cloud Computing*, 2021.

[33] WebAssembly Community Group. “WebAssembly Threads Proposal,” Accessed: Mar. 23, 2026. [Online]. Available: https://github.com/WebAssembly/threads

[34] A. Spies and M. Mock, “An evaluation of WebAssembly in non-web environments,” in *Proceedings of the 14th ACM International Conference on Systems and Storage*, 2021, pp. 1–11.

[35] A. Jangda, B. Powers, E. D. Berger, and A. Guha, “Not So Fast: Analyzing the Performance of WebAssembly vs. Native Code,” in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, 2019, pp. 107–120.

[36] T. Combe, A. Martin, and R. Di Pietro, “To Docker or Not to Docker: A Security Perspective,” *IEEE Cloud Computing*, vol. 3, no. 5, pp. 54–62, 2016.

[37] D. Lehmann, J. Kinder, and M. Pradel, “Everything Old is New Again: Binary Security of WebAssembly,” in *29th USENIX Security Symposium (USENIX Security 20)*, 2020, pp. 217–234.

[38] Y. Zhang, M. Liu, H. Wang, Y. Ma, G. Huang, and X. Liu, “Research on WebAssembly Runtimes: A Survey,” *ACM Transactions on Software Engineering and Methodology*, 2024, Also available as arXiv:2404.12621. DOI: 10.1145/3714465

[39] S. Kakati and M. Brorsson, “A Cross-Architecture Evaluation of WebAssembly in the Cloud-Edge Continuum,” in *2024 IEEE 24th International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, IEEE, 2024.

[40] J. A. Wiegratz, “Comparing Security and Efficiency of WebAssembly and Linux Containers in Kubernetes Cloud Computing,” *arXiv preprint arXiv:2411.03344*, 2024. DOI: 10.48550/arXiv.2411.03344

[41] V. Kjorveziroski and S. Filiposka, “WebAssembly Orchestration in the Context of Serverless Computing,” *Journal of Network and Systems Management*, vol. 31, no. 62, 2023. DOI: 10.1007/s10922-023-09753-0

[42] Cloudflare, Inc. “Cloudflare Workers: Serverless Execution Environment,” Accessed: Mar. 24, 2026. [Online]. Available: https://workers.cloudflare.com/

[43] Fastly, Inc. “Fastly Compute: Edge Computing Platform,” Accessed: Mar. 24, 2026. [Online]. Available: https://www.fastly.com/products/edge-compute

[44] Fastly Engineering. “How Lucet and Wasmtime Make a Stronger Compiler, Together,” Fastly, Accessed: Mar. 25, 2026. [Online]. Available: https://www.fastly.com/blog/how-lucet-wasmtime-make-stronger-compiler-together

[45] Fermyon Technologies. “Fermyon Spin: The Developer Tool for Serverless WebAssembly,” Accessed: Mar. 23, 2026. [Online]. Available: https://developer.fermyon.com/spin/v3/index

[46] WasmCloud Project. “WasmCloud: A Platform for Writing Portable Business Logic,” Accessed: Mar. 24, 2026. [Online]. Available: https://wasmcloud.com/

[47] SUSE Rancher. “K3s: Lightweight Kubernetes,” Accessed: Mar. 23, 2026. [Online]. Available: https://k3s.io/

[48] CNCF TAG Runtime. “WebAssembly Working Group,” Cloud Native Computing Foundation, Accessed: Mar. 25, 2026. [Online]. Available: https://tag-runtime.cncf.io/wgs/wasm/

[49] A. Musaev. “thesis-experiments: Benchmark implementations, runners, and result data for the master’s thesis,” GitHub, Accessed: May 18, 2026. [Online]. Available: https://github.com/abdulaziz7225/thesis-experiments

[50] HashiCorp. “Terraform: Infrastructure as Code,” Accessed: Mar. 23, 2026. [Online]. Available: https://www.terraform.io/

[51] Prometheus Authors. “Prometheus: Monitoring System and Time Series Database,” Accessed: Mar. 23, 2026. [Online]. Available: https://prometheus.io/

[52] Rust Project Team. “The wasm32-wasip2 Target Has Reached Tier 2 Support,” Rust Blog, Accessed: May 18, 2026. [Online]. Available: https://blog.rust-lang.org/2024/11/26/wasip2-tier-2/

[53] Grafana Labs. “k6: Open-Source Load Testing Tool,” Accessed: Mar. 23, 2026. [Online]. Available: https://k6.io/

[54] A. Musaev. “thesis-infra-setup: Kubernetes/SpinKube cluster provisioning code for the master’s thesis benchmarks,” GitHub, Accessed: May 18, 2026. [Online]. Available: https://github.com/abdulaziz7225/thesis-infra-setup

[55] Hetzner Online GmbH. “Cloud Servers — Hetzner Online,” Accessed: Mar. 23, 2026. [Online]. Available: https://www.hetzner.com/cloud/

[56] J. Ménétrey, M. Pasin, P. Felber, and V. Schiavoni, “Twine: An Embedded Trusted Runtime for WebAssembly,” in *37th IEEE International Conference on Data Engineering (ICDE)*, 2021, pp. 205–216. DOI: 10.1109/ICDE51399.2021.00025

[57] J. Ménétrey, M. Pasin, P. Felber, and V. Schiavoni, “WaTZ: A Trusted WebAssembly Runtime Environment with Remote Attestation for TrustZone,” in *42nd IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2022, pp. 1177–1189. DOI: 10.1109/ICDCS54860.2022.00116

[58] D. Goltzsche, M. Nieke, T. Knauth, and R. Kapitza, “AccTEE: A WebAssembly-based Two-way Sandbox for Trusted Resource Accounting,” in *Proceedings of the 20th International Middleware Conference (Middleware ’19)*, 2019, pp. 123–135. DOI: 10.1145/3361525.3361541

[59] Confidential Computing Consortium. “Enarx: Confidential Computing with WebAssembly,” Confidential Computing Consortium / Linux Foundation, Accessed: May 25, 2026. [Online]. Available: https://enarx.dev/

[60] Confidential Containers Project. “Confidential Containers: Cloud-native confidential computing for Kubernetes,” Cloud Native Computing Foundation, Accessed: May 25, 2026. [Online]. Available: https://confidentialcontainers.org/

[61] A. Musaev. “master-thesis: LaTeX source of “A Comparative Analysis of WebAssembly and Docker Containers for Microservice Architectures in Kubernetes”,” GitHub, Accessed: May 18, 2026. [Online]. Available: https://github.com/abdulaziz7225/master-thesis